US Patent & Trademark Office
Patent Public Search | Text View

United States Patent	12405979
Kind Code	B2
Date of Patent	September 02, 2025
Inventor(s)	Madisetti; Vijay et al.

Method and system for optimizing use of retrieval augmented generation pipelines in generative artificial intelligence applications

Abstract

A system and method of improving performance of LLMs including receiving a query, retrieving relevant documents for the query to form a combined context, partitioning the combined context in context partitions, generating intermediate results from the context partitions using a mapper prompt, and generating a final result from the intermediate results using a reducer prompt. The final result is transmitted to a user.

Inventors: **Madisetti; Vijay (Alpharetta, GA), Bahga; Arshdeep (Chandigarh, IN)**

Applicant: **Madisetti; Vijay** (Alpharetta, GA)

Family ID: **95939976**

Assignee: **Madisetti; Vijay (Alpharetta, GA)**

Appl. No.: **19/056496**

Filed: **February 18, 2025**

Prior Publication Data

Document Identifier	Publication Date
US 20250190461 A1	Jun. 12, 2025

Related U.S. Application Data

continuation parent-doc US 18348692 20230707 US 12001462 20240604 child-doc US 18470487
continuation-in-part parent-doc US 19040471 20250129 PENDING child-doc US 19056496
continuation-in-part parent-doc US 18921852 20241021 PENDING child-doc US 19040471
continuation-in-part parent-doc US 18812707 20240822 PENDING child-doc US 18921852
continuation-in-part parent-doc US 18470487 20230920 US 12147461 20241119 child-doc US 18812707

us-provisional-application US 63742792 20250107
us-provisional-application US 63693351 20240911
us-provisional-application US 63647092 20240514
us-provisional-application US 63607647 20231208
us-provisional-application US 63607112 20231207
us-provisional-application US 63535118 20230829
us-provisional-application US 63534974 20230828
us-provisional-application US 63529177 20230727
us-provisional-application US 63469571 20230530
us-provisional-application US 63463913 20230504

Publication Classification

Int. Cl.: **G06F16/3329** (20250101); **G06F40/284** (20200101)

U.S. Cl.:

CPC **G06F16/3329** (20190101); **G06F40/284** (20200101);

Field of Classification Search

CPC: G06F (16/3329); G06F (40/284)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
5410475	12/1994	Lu et al.	N/A	N/A
8775436	12/2013	Zhou	707/748	G06V 30/416
11194868	12/2020	Paka	N/A	G06F 16/9535
11928569	12/2023	Douthit	N/A	N/A
11995411	12/2023	Qadrud-Din	N/A	G06F 40/40
12242503	12/2024	Kelsey et al.	N/A	N/A
2007/0244937	12/2006	Flynn, Jr. et al.	N/A	N/A
2008/0077569	12/2007	Lee	707/999.005	G06F 16/358
2010/0138531	12/2009	Kashyap	N/A	N/A
2012/0023073	12/2011	Dean et al.	N/A	N/A
2012/0290521	12/2011	Frank et al.	N/A	N/A
2014/0297845	12/2013	Tamura	N/A	N/A
2018/0095845	12/2017	Sanakkayala et al.	N/A	N/A
2019/0081959	12/2018	Yadav et al.	N/A	N/A
2019/0130006	12/2018	Raviv	N/A	G06F 3/0486
2019/0130902	12/2018	Itoh et al.	N/A	N/A
2022/0124013	12/2021	Chitalia et al.	N/A	N/A
2024/0289560	12/2023	Kelly et al.	N/A	N/A
2024/0354130	12/2023	Cadoni et al.	N/A	N/A
2024/0354490	12/2023	Chauvin et al.	N/A	N/A
2024/0404687	12/2023	Bell et al.	N/A	N/A
2024/0412029	12/2023	Yang et al.	N/A	N/A

2025/0036878	12/2024	Marwah	N/A	G06F 40/30
2025/0045256	12/2024	Gottlob	N/A	G06F 16/215
2025/0068665	12/2024	Chandel	N/A	G06F 16/338

OTHER PUBLICATIONS

Non Final Office Action received in related U.S. Appl. No. 19/040,471 issued on Apr. 9, 2025; 11 pages. cited by applicant
Notice of Allowance received in related U.S. Appl. No. 18/812,707 issued on May 20, 2025; 26 pages. cited by applicant
Notice of Allowance received in related U.S. Appl. No. 18/921,852 issued Jun. 20, 2025; 25 pages. cited by applicant

---

Primary Examiner: Yen; Eric

Attorney, Agent or Firm: Widerman Malek, PL

Background/Summary

RELATED APPLICATIONS (1) This application is a continuation-in-part application of and claims priority under 35 U.S.C. § 120 of U.S. patent application Ser. No. 19/040,471 filed on Jan. 29, 2025 and titled Method and System for Multi-Level Artificial Intelligence Supercomputer Design, which in turn claims priority under 35 U.S.C. § 119 (e) of U.S. Provisional Patent Application Ser. No. 63/742,792 No. 3026.00206) filed on Jan. 7, 2025 and titled Hierarchical Tokens for Fine Tuning and Model Training and is also a continuation-in-part application of and claims priority under 35 U.S.C. § 120 of U.S. patent application Ser. No. 18/921,852, filed on Oct. 21, 2024, and titled Fault Tolerant Multi-Agent Generative AI Applications, which in turn claims priority under 35 U.S.C. § 119 (e) of U.S. Provisional Patent Application Ser. No. 63/693,351, filed on Sep. 11, 2024, and titled Fault Tolerant MultiAgent Generative AI Applications. The '852 application is also a continuation-in-part application of and claims priority under 35 U.S.C. § 120 of U.S. patent application Ser. No. 18/812,707, filed on Aug. 22, 2024, and titled Method and Systems for Optimizing User of Retrieval Augmented Generation Pipelines in Generative Artificial Intelligence Applications, which in turn is a continuation-in-part application of and claims priority under 35 U.S.C. § 120 of U.S. patent application Ser. No. 18/470,487, now U.S. Pat. No. 12,147,461, issued Nov. 19, 2024, filed on Sep. 20, 2023, and titled Method and System for Multi-Level Artificial Intelligence Supercomputer Design, which in turn is a continuation application of and claims priority under 35 U.S.C. § 120 of U.S. patent application Ser. No. 18/348,692, now U.S. Pat. No. 12,001,462, issued Jun. 4, 2024, filed on Jul. 7, 2023, and titled Method and System for Multi-Level Artificial Intelligence Supercomputer Design, which in turn claims priority under 35 U.S.C. § 119 (e) of U.S. Provisional Patent Application Ser. No. 63/463,913, filed on May 4, 2023, and titled New Tools for Document Analysis in CatchUp and of U.S. Provisional Patent Application Ser. No. 63/469,571, filed on May 30, 2023, and titled Multilevel AI PSupercomputer Design. The '707 application also claims priority under 35 U.S.C. § 119 (e) of U.S. Provisional Patent Application Ser. No. 63/535,118, filed on Aug. 29, 2023, and titled Networked LLMs and Focused LLMs, U.S. Provisional Patent Application Ser. No. 63/529,177, filed on Jul. 27, 2023, and titled Using LLMs to Create Projects and Tasks in an Optimized Way, U.S. Provisional Patent Application Ser. No. 63/534,974, filed on Aug. 28, 2023, and titled Using Prompts to Generate Search Queries for Context Generation in LLMs, U.S. Provisional Patent Application Ser. No. 63/647,092, filed on May 14, 2024, and titled Using LLMs to Influence Users and Organizations, U.S. Provisional Patent Application Ser. No. 63/607,112, filed on Dec. 7, 2023, and titled Long

Document Attention Span Enhancement for LLMs, and U.S. Provisional Patent Application Ser. No. 63/607,647, filed on Dec. 8, 2023, and titled High-Level UI for Prompt Generation for LLMs. The contents of these applications are incorporated herein by reference.

FIELD OF THE INVENTION

(1) The present invention primarily relates to artificial intelligence and large language models (LLMs) for generative AI applications.

BACKGROUND

(2) Large Language Models (LLMs) are generative Artificial Intelligence (AI) models which are trained on limited amounts of data and can perform language processing tasks (with multimodal inputs-text, and more recently, image inputs as in Microsoft's Kosmos-1) and generate human-like text (and associated multimedia material, like images, video and advertisements). LLMs have many parameters (from millions to billions). LLMs can capture complex patterns in language and produce text that closely resembles human language.

(3) The high-level goal of an LLM is to predict the text (and other multimedia material) that is likely to come next in a sequence. The applicants recognize that LLMs are a type of generative AI that is in usually different from traditional machine learning and AI applications. LLM also stands for Learning with Limited Memory and implies that LLM's are closely tied to their training data and make decisions based on the limited amount of data. Both generative AI and LLM generate content, but LLM does it in a manner that improves computational and memory efficiency.

(4) Traditional machine learning type algorithms focus on analysis, such as statistical regression or clustering, and are usually again different from Generative AI and LLMs, which focus on generating content. LLMs have immediate practical implication in generation of new content that matches associated or preceding/future content in an optimized manner, such as legal briefs or computer code, based on training with a limited amount of data, such as existing briefs or code, both from private and public sources. In this invention, we focus on LLM models as the primary focus of these improvements, though we do not disclaim other AI models, unless expressly done as part of the claims.

(5) LLMs are created with complex architectures such as transformers, encoders and decoders. LLMs, typically, use a technique of natural language processing called Tokenization that involves splitting the input text (and images) and output texts into smaller units called tokens. Tokens can be words, characters, sub-words, or symbols, depending on the type and the size of the model. Tokenization helps to reduce the complexity of text data, making it easier for LLMs to process and understand data thus reducing the computational and memory costs. Another important component of an LLM is Embedding, which is a vector representation of the tokens. The Encoder, within the Transformer architecture, processes the input text and converts it into a sequence of vectors, called embeddings, that represent the meaning and context of each word. The Decoder, within the Transformer architecture, generates the output text by predicting the next word in the sequence, based on the embeddings and the previous words. LLMs use Attention mechanisms that allow the models to focus selectively on the most relevant parts of the input and output texts, depending on the context of the task at hand, thus capturing the long-range dependencies and relationships between words.

(6) LLMs are designed to learn the complexity of the language by being pre-trained on vast amounts of text (and multimedia) data from sources such as Wikipedia, books, articles on the web, social media data and other sources. The training procedure can be decomposed into two stages: 1. Pre-training on a large amount of unlabeled plain text; and 2. Supervised fine-tuning

(7) Through training on limited amounts of data, the models are able to learn the statistical relationships between words, phrases, and sentences and other multimedia content. The trained models can then be used for generative AI applications such as Question Answering, Instruction Following, Inferencing, for instance, where an input is given to the model in the form of a prompt

and the model is able to generate coherent and contextually relevant responses based on the query in the prompt.
(8) Popular LLM models include GPT (Generative Pre-trained Transformer), BERT (Bidirectional Encoder Representations from Transformers), BART (Bidirectional and Auto-Regressive Transformers) and PaLM (Pathways Language Model). See, for example, public domain websites, such as openai.com or bard.google.com for more information as to how a person of ordinary skill in the art may use these models. Public domain and company-specific LLMs, such as GPT4AII, MiniGPT4, RMKV, BERT, MPT-7B, Kosmos-1 (which accepts image and multimodal inputs), YaLM, are also available for wide use, as for example, described in medium.datadriveninvestor.com/list-of-open-source-large-language-models-llms-4eac551bda2e.
(9) Current AI generative models and LLMs require super-computing efforts to compute results and an efficient way to improve response times, accuracies, and reduce computational load is required to improve both cost and scalability and expandability of existing AI models and their use.
(10) LLMs face significant challenges when processing long documents, particularly in maintaining coherence and performing long-range reasoning. This limitation, often referred to as the “attention span problem,” causes a noticeable drop in performance as the length of the input context increases, typically above 10,000 to 50,000 tokens.
(11) The attention span problem has substantial implications for real-world applications, especially in domains that frequently deal with lengthy and complex documents, such as legal, engineering, healthcare, and academic research. In these fields, the ability to comprehend and reason over extended contexts is crucial for tasks like document summarization, question answering, and information extraction.
(12) Existing approaches to mitigate the attention span problem, such as sliding window techniques or hierarchical attention mechanisms, have shown limited success. They often struggle to maintain global coherence or fail to capture long-range dependencies effectively. As a result, there is a pressing need for innovative solutions that can enhance the attention span of LLMs and enable them to process long documents more effectively.
(13) LLMs face inherent limitations due to their reliance on pre-trained knowledge. These include a fixed knowledge cutoff, potential for hallucination, and lack of specificity in responses. Retrieval-Augmented Generation (RAG) is a useful approach in AI that combines the strengths of LLMs with external knowledge retrieval. RAG addresses the limitations of LLMs by providing them with relevant, up-to-date information from a curated knowledge base. This approach grounds LLM outputs in retrieved facts, significantly reducing hallucinations while enabling more accurate and context-specific responses.
(14) Existing RAG systems have shown promise in enhancing the performance of LLMs by providing relevant context from external knowledge sources. However, these systems face significant challenges in processing and retrieving information from long, complex documents. Current RAG implementations often struggle with inefficient document chunking, leading to loss of context and semantic coherence. They typically rely on simplistic keyword-based retrieval methods, which fail to capture the nuanced graph-like relationships between concepts. Moreover, existing systems lack sophisticated mechanisms for dynamically adapting to different types of queries and documents, resulting in sub-optimal retrieval and generation performance. The inability to effectively handle large volumes of text, combined with inadequate context preservation and limited semantic understanding, hinders the widespread adoption of RAG systems in domains that deal with extensive and intricate textual information, such as legal, medical, engineering, and scientific research fields.
(15) LLMs face significant challenges in managing context windows and maintaining semantic coherence across long documents. While existing approaches like Retrieval Augmented Generation (RAG) and Hierarchical Navigable Small World (HNSW) indexes have made progress in addressing these challenges, they still operate primarily at the token level, which can limit their

effectiveness in capturing and utilizing higher-level functional relationships within text.
(16) LLMs currently face challenges in understanding causality, and combining correlation with causation in the responses. While LLMs can extract and recite causal relationships from their training data, they struggle to genuinely reason about cause and effect, leading to potential errors in tasks requiring causal understanding. This limitation is particularly problematic in domains like healthcare, legal analysis, and policy-making, where understanding true causal relationships is crucial for making informed decisions.
(17) This background information is provided to reveal information believed by the applicant to be of possible relevance to the present invention. No admission is necessarily intended, nor should be construed that any of the preceding information constitutes prior art against the present invention.
SUMMARY OF THE INVENTION
(18) With the above in mind, embodiments of the present invention are directed to a system and associated methods for multi-level generative AI and large language models (LLM) for generative AI applications, that utilize the following techniques:
(19) Derived Requests: An initial level of generative AI software program, or AI broker, evaluates the incoming client request (maybe a conversational query or through an API, such as OpenAI API) and identifies its specific AI “characteristics” that may make it suitable for one or other or both or multiple AI language models and checks its “derived requests” categories to see if the query suits one of the “derived requests” categories and/or it can or should create a new request.
(20) Multiple h-LLMs: If the new request does is not assigned to one or more of the “derived requests) categories, it evaluates the request and selects one or more AI h-LLM model categories for its evaluation. An h-LLM is a family of models, such as GPT-4, that (in addition) have been trained according to a particular training set T**1**. A family of generative models, LLM**1**, trained with a data set T**1**, can be represented as h-LLM**1**, while a family of models, LLM**2**, trained with data set T**2**, can be represented as h-LLM**12**. Further, a family of models, LLM**1**, trained with a data set T**3**, can be represented as h-LLM**35**. The combination of models and their training sets (T**1** could be a subset of T**3**, for example, or they can be different) may be used in our proposed invention and they are referred to as h-LLMs, throughout. A family of LLMs that operate at a lower arithmetic precision, on computer CPUs or graphical processing units (GPUs, such as Nvidia's H100), may also be called by a different identifier, e.g., h-LLM**14**, when trained with its corresponding data set.
(21) Choosing h-LLMs with varying levels of accuracy: It further checks the workload of the AI h-LLM models in the one or more categories and its level of training and its accuracy-called its workload scores or its technical accuracy scores, or its business value metrics or a combination of these scores, and then assigns the request (or its derived form) to one or more of the AI h-LLM models within the selected AI h-LLM model categories.
(22) Assigning weights to results: It then receives the results from the AI models in the AI h-LLM models categories and weights them to compute a result that could be returned to the requester program, or it could resend the request back to the AI h-LLM models/categories hierarchy till it reaches a certain level of service level assurance.
(23) Use of Local Database: It also updates a local database with the results of the request's path through its hierarchy and create an index of “derived requests” that may be used in future to select which set of “derived requests” an incoming request may fall into for further processing.
(24) Distributed Architecture: The tasks may be implemented as containers within Kubernetes environment and a service mesh, such as Istio, may be used to instrument and parameterize the metrics and log collections, but not limited to these cloud models for implementation.
(25) Efficient Search & Retrieval: Traditional online and offline approaches to cluster search are used to find the relevant subset of the documents being evaluated in Retrieval Augmented Generation (RAG) pipelines. Once this subset is retrieved then the traditional pipeline of LLMs operations are carried out as in LangChain and LlamaIndex. The cluster may be generated during time of the Query Prompt input (adds to the delay due to need to generate indexes) or could be used

to select a subset of indexes in Vector Db in a quicker approach. Few important queries and prompts may be used to generate clusters (offline) and each new online query may be mapped to the best "cluster" that was pre-generated based on that query or similar queries.
(26) Network of LLMs working together to replace a larger LLM: Currently a single large LLM is trained on all types of data and has large number of parameters (e.g. OpenAI GPT3.5 has 175 billion parameters and GPT-4 has over 1 trillion of parameters). A approach using a Network of LLMs is proposed which combines smaller LLMs (with 3B or 7B parameters, for example), preferably each focused on a specific type of result (cost estimation, profit estimation, expense estimation or prediction). The network of LLMs is used to provide a composite result that is easier to prompt for, easier to optimize and easier to "explain" how it works by having smaller focused LLMs trained on specialized training sets.
(27) Embodiments of the present invention are directed to a system and associated methods for enhancing the attention span of Large Language Models (LLMs) when processing long documents. The system, long-document attention span enhancement through refinement ("LASER"), uses an iterative attention focusing technique that dynamically refines and condenses document and chunk context to improve model comprehension and coherence over extended inputs.
(28) Other embodiments of the present invention are directed to enhancing Retrieval-Augmented Generation (RAG) through context-optimized retrieval techniques. The system, scored context-optimized retrieval enhancement for retrieval augmented generation ("SCORE-RAG"), addresses the limitations of existing RAG systems by incorporating advanced document (including chunk) processing, intelligent information retrieval, and adaptive response generation mechanisms.
(29) In one embodiment, the present invention comprises a document processing system that includes an input module, a model module, an iteration controller, a knowledge module, and an output handler. The input module is configured to split long documents into manageable blocks (or chunks) and generate iterative contexts, while the model module contains an attention model for processing these contexts and a ranking unit for evaluating outputs.
(30) Another embodiment of the invention involves a method for iterative attention focusing, which includes splitting a long document into blocks, batching these blocks, processing them through an LLM, ranking the outputs, and then clustering and reforming new batches based on the highest-ranked content. This process is repeated iteratively, gradually condensing the document to its most relevant parts.
(31) Another embodiment of the invention provides a mechanism for dynamically adjusting the attention focus of an LLM. This mechanism employs a ranking system that scores model outputs based on coherence and relevance, allowing the system to identify and prioritize the most important parts of a document or chunks across multiple processing cycles.
(32) Another embodiment of the invention introduces a knowledge module that incorporates an extractive summarizer and a document clustering component. These elements work together to identify key information and group related content, further enhancing the system's ability to distill and focus on critical parts of long documents and chunks.
(33) Another embodiment of the invention comprises a document processor for handling various input formats, a topic modeling engine for semantic analysis, and an intelligent document chunking module that preserves contextual integrity. This embodiment also features a hybrid search module that combines keyword-based and vector similarity search methods for improved retrieval accuracy.
(34) Another embodiment of the invention involves a method for dynamically processing and indexing documents. This method employs a citation analyzer to assess the importance of different text segments, a chunk selection and ranking module to identify the most relevant portions of a document, and a metadata enrichment module to enhance the contextual information associated with each text chunk. The method further includes an adaptive indexing process that optimizes storage and retrieval of processed information.

(35) Another embodiment of the invention provides a mechanism for query augmentation and response generation. This mechanism utilizes a Query Processor to analyze and classify user inputs, an Augmentation Engine to integrate retrieved context with the original query, and a Generation Module that interfaces with LLMs to produce coherent and relevant responses.
(36) Embodiments of the present invention are directed to a system and associated methods for enhancing LLMs through Functional Language Modeling (FLM) that utilize Hierarchical Tokens (H-Tokens) to improve context management and semantic coherence. The system addresses limitations in traditional token-based processing by introducing functional abstractions that can compress multiple regular tokens into single semantic units while preserving functional meaning.
(37) In one embodiment, the present invention comprises a document processing system that includes an input processing module configured to recognize document domains and identify functional components, a function identification module for analyzing and mapping hierarchical relationships between functions, and an H-Token generation module that encapsulates identified functions into compressed token representations. The system further includes implementation modules for mapping H-Tokens to specific functions, events, and implementation methods within their respective domains.
(38) Another embodiment of the invention involves a method for processing domain-specific content through functional abstraction, which includes analyzing input documents to identify domain-specific functions, breaking these functions into sub-functions, generating H-Tokens that encapsulate the functional units, and processing these H-Tokens either through expansion to regular tokens or direct H-Token processing in conjunction with RAG systems. This method enables efficient handling of long documents while maintaining semantic coherence across document boundaries.
(39) Another embodiment of the invention provides a mechanism for dynamically adapting the FLM system across different domains (such as legal, travel, customer service, technical, for instance). This mechanism employs domain-specific analyzers to identify relevant functions, generates appropriate H-Tokens for each domain, and maintains relationships between functions, events, and implementation methods. The mechanism can process multi-modal inputs and integrate with existing RAG systems through either token expansion or direct H-Token processing paths, providing flexibility in implementation while improving context management efficiency.
(40) Another embodiment of the invention introduces a hybrid processing approach that combines H-Token based processing with traditional RAG systems, enabling organizations to leverage existing infrastructure while gaining the benefits of functional abstraction. This approach includes methods for H-Token context assembly, processing path selection, and output generation that can be customized based on specific domain requirements and processing needs.
(41) Embodiments of the present invention are directed to a system and associated methods for processing large contexts in a retrieval-augmented generation (RAG) system using a multi-pass approach that combines document retrieval, parallel processing, and sophisticated response generation.
(42) In one embodiment, the present invention comprises a hybrid search system that combines keyword-based searching in a relational database with vector-based searching in a vector database to identify and retrieve relevant documents, followed by de-duplication to form a combined context.
(43) Another embodiment of the invention provides a mechanism for parallel processing of large document sets by partitioning the combined context into multiple portions while preserving document boundaries, with each portion being processed concurrently by separate instances of a large language model using specialized mapper prompts.
(44) Another embodiment of the invention involves a method for processing intermediate analysis results using a reducer component that identifies common themes, resolves conflicts, and synthesizes information into a coherent response while maintaining accuracy and completeness.

(45) In another embodiment, the invention includes a caching system for storing intermediate results, enabling quick responses to subsequent related queries without full reprocessing of previously analyzed documents.
(46) Another embodiment provides a comprehensive analysis system that extracts key information, generates summaries, and identifies relationships within each context portion, storing these elements as part of the intermediate results.
(47) In yet another embodiment, the invention implements a weighted combination approach where intermediate results are assigned confidence scores that influence their impact on the final response generation.
(48) Another embodiment of the invention comprises a resource management system that continuously monitors computational capacity and dynamically adjusts context portion sizes to optimize processing efficiency.
(49) In a further embodiment, the invention employs specialized language models for different processing phases, including a first LLM for analysis and extraction, a second LLM for synthesis and summarization, and a third LLM for fact-checking and verification.
(50) Another embodiment of the invention maintains a conversation history that is incorporated into the reducer component, enabling generation of contextually appropriate responses across multiple user interactions.
(51) In another embodiment, the invention provides a modular architecture that allows for different combinations of databases, language models, and processing architectures while maintaining the core mapper-reducer approach to context processing.
(52) Embodiments of the present invention are directed to a system and associated methods for incorporating probabilistic causal analysis into retrieval-augmented generation (RAG) systems, enabling more accurate identification and reasoning about cause-effect relationships while maintaining uncertainty quantification throughout the processing pipeline.
(53) In one embodiment, the present invention comprises a probabilistic causal mapper that extracts events and their temporal relationships from document contexts while assigning probability distributions to potential causal links between events. The mapper employs specialized LLMs to identify key events, their constituent entities, and calculate confidence scores for each extracted element, while preserving document boundaries and semantic coherence within each context partition.
(54) Another embodiment of the invention relates to a method for processing intermediate causal analysis results using a probabilistic reducer component that constructs multiple potential causal chains, each with an associated probability score. The reducer calculates compound probabilities for causal chains by combining individual link probabilities and confidence scores, while collecting supporting evidence for each proposed causal relationship.
(55) In another embodiment, the invention provides a hybrid search system that combines keyword-based and vector-based searching to identify relevant documents, followed by probabilistic causal analysis to extract and validate potential cause-effect relationships within the retrieved context.
(56) Another embodiment of the invention implements a weighted causal chain aggregation approach where multiple possible causal explanations are maintained simultaneously, each with associated probability scores that influence their impact on the final response generation.
(57) In yet another embodiment, the invention employs specialized language models for different phases of causal analysis, including a first LLM optimized for event extraction and initial causal link identification, a second LLM for probability estimation and causal chain construction, and a third LLM for validation and evidence collection to support proposed causal relationships.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) FIG. **1** is an illustration of the training process for creating multiple specialized large language models for specific tasks/categories, according to an embodiment of the present invention.

(2) FIG. **2** is an illustration of h-LLMs trained with different training sets, according to an embodiment of the invention.

(3) FIG. **3** is an illustration of the process for generating synthetic data from multiple h-LLMs and using it for model refinement, according to an embodiment of the invention.

(4) FIG. **4** is an illustration of a “bagging” approach where multiple h-LLMs with lower precision and accuracy are merged/fused to create a merged h-LLM with higher precision and accuracy, according to an embodiment of the invention.

(5) FIG. **5** is an illustration of a “boosting” approach where multiple h-LLMs of increasing precision and accuracy are created in a sequential manner and then merged/fused to create a merged h-LLM, according to an embodiment of the invention.

(6) FIG. **6** is an illustration of creating a smaller and more specialized h-LLM through extraction/specialization process from a larger h-LLM, according to an embodiment of the invention.

(7) FIG. **7** is an illustration of combining h-LLMs trained with text, image and audio data to create a merged h-LLM, according to an embodiment of the invention.

(8) FIG. **8** is an exemplary illustration of an application of using AI models for detecting labels in PDF files, according to an embodiment of the invention.

(9) FIG. **9** is an illustration of generating derived prompts for different categories and using them with multiple h-LLMs to generate the best results, according to an embodiment of the present invention.

(10) FIG. **10** is an illustration of using multiple h-LLMs to answer questions from specific input documents, according to an embodiment of the present invention.

(11) FIG. **11** is an illustration of an AI Broker for processing results from multiple h-LLMs, according to an embodiment of the present invention.

(12) FIG. **12** is an illustration of the combining h-LLMs in series, according to an embodiment of the present invention.

(13) FIG. **13** is an illustration of combining h-LLMs in parallel, according to an embodiment of the present invention.

(14) FIG. **14** is an illustration of a hybrid approach of combining h-LLMs in series and parallel, according to an embodiment of the present invention.

(15) FIG. **15** is an illustration of the lambda architecture for h-LLMs, according to an embodiment of the present invention.

(16) FIG. **16** is an illustration of batch and real-time processing architecture for h-LLMs, according to an embodiment of the present invention.

(17) FIG. **17** is an illustration of an in-memory processing architecture for h-LLMs, according to an embodiment of the present invention.

(18) FIG. **18** is an illustration of the architecture of PDF label search tool with CatchUp GlassViewer, according to an embodiment of the invention.

(19) FIG. **19** is an exemplary interface of the CatchUp platform showing the document management system, according to an embodiment of the invention.

(20) FIG. **20** is an exemplary interface of the CatchUp platform showing the PDF viewer (GlassViewer), according to an embodiment of the invention.

(21) FIG. **21** is an exemplary interface of the CatchUp platform showing a magnifier tool within the GlassViewer for searching labels, according to an embodiment of the invention.

(22) FIG. **22** is an exemplary interface of the CatchUp platform showing label search results within GlassViewer, according to an embodiment of the invention.

(23) FIG. **23** is an illustration of the process used by organizations to decide the list of tasks for a project, according to an embodiment of the invention.
(24) FIG. **24** is an illustration of the process of training LLMs to generate project, tasks and action items, according to an embodiment of the invention.
(25) FIG. **25** is an illustration of the process of creating a project within CatchUp using a prompt, according to an embodiment of the invention.
(26) FIG. **26** is an illustration of the process of Retrieval Augmented Generation (RAG), according to an embodiment of the invention.
(27) FIG. **27** is an illustration of the process of indexing documents RAG, according to an embodiment of the invention.
(28) FIG. **28** is an illustration of the process of querying documents in RAG, according to an embodiment of the invention.
(29) FIG. **29** is an illustration of the Retrieval Augmented Generation RAG pipeline, according to an embodiment of the invention.
(30) FIG. **30** is an illustration of an approach that uses multiple RAG pipelines with clusters of documents, according to an embodiment of the invention.
(31) FIG. **31** is an illustration of an approach that uses a prompt augmented with a search query, according to an embodiment of the invention.
(32) FIG. **32** is an illustration of an approach that uses a search query to extract relevant context for RAG pipelines, according to an embodiment of the invention.
(33) FIG. **33** is an exemplary interface of the CatchUp platform for RAG, according to an embodiment of the invention.
(34) FIG. **34** is an exemplary interface of the CatchUp platform showing response a RAG pipeline, according to an embodiment of the invention.
(35) FIG. **35**A is an illustration of a prior art embodiments of using a single LLM.
(36) FIG. **35**B is an illustration of using a network of LLMs, according to an embodiment of the invention.
(37) FIGS. **36**A-C is a comparison of the current (**36**A) and proposed (**36**B, **36**C) approaches of a long-document attention span enhancement through refinement (LASER) system and a scored context-optimized retrieval enhancement for retrieval augmented generation (SCORE-RAG) system according to an embodiment of the invention.
(38) FIG. **37** is an illustration of the high-level architecture of a LASER system according to an embodiment of the invention.
(39) FIG. **38** is a flow chart illustrating a method performed by a LASER system by which long documents are processed and refined to enhance the attention span of LLMs according to an embodiment of the invention.
(40) FIG. **39** is a flow chart illustrating a method of adapting a MapReduce model for the LASER system according to an embodiment of the invention.
(41) FIG. **40** is an illustration of the architecture of a SCORE-RAG system according to an embodiment of the invention.
(42) FIG. **41** is a flow chart illustrating a method of a SCORE-RAG system according to an embodiment of the invention.
(43) FIG. **42** is an illustration of an implementation of a SCORE-RAG system according to an embodiment of the invention for multi-modal RAG.
(44) FIG. **43** is an illustration of an implementation of a LASER system and a SCORE-RAG system according to an embodiment of the invention for multi-modal RAG.
(45) FIG. **44** is an illustration of different meanings of “document” in the context of the present invention.
(46) FIG. **45** is an illustration of different meanings of “chunk” and “block” in the context of the present invention.

(47) FIG. **46** is an illustration of different meanings of “meta-data” in the context of the present invention.
(48) FIG. **47** is an illustration of different meanings of “document processing” or “chunk/block processing” in the context of the present invention.
(49) FIG. **48** is an illustration of an approach of generating derived prompts and using them with RAG or SCORE-RAG systems to generate improved results according to an embodiment of the invention.
(50) FIG. **49** is an illustration of an approach of generating derived prompts and using LASER and RAG or SCORE-RAG systems to generate improved results according to an embodiment of the invention.
(51) FIG. **50** is an illustration of an exemplary set of APIs of a LASER system according to an embodiment of the invention.
(52) FIG. **51** is an illustration of an exemplary set of APIs of the SCORE-RAG system, according to an embodiment of the invention.
(53) FIG. **52** is a flowchart illustrating methods of performing prefill and decode phases of an LLM inference process according to an embodiment of the invention.
(54) FIG. **53** is an illustration depicting the formation of a superchunk from several chunks from discrete documents according to an embodiment of the invention.
(55) FIG. **54** is an illustration of different meanings of “superchunk” in the context of the present invention.
(56) FIG. **55** is an illustration of characteristics of superchunks according to an embodiment of the invention.
(57) FIG. **56** is an illustration of the architecture of a Hybrid-RAG system according to an embodiment of the invention.
(58) FIG. **57** is an illustration of an architecture of a NoRAG system according to an embodiment of the invention.
(59) FIG. **58** is an illustration of the H-Token components in the FLM system according to an embodiment of the invention.
(60) FIG. **59** is an illustration of the steps in the FLM system according to an embodiment of the invention.
(61) FIG. **60** is an illustration of FLM system implementation in the travel domain according to an embodiment of the invention.
(62) FIG. **61** is an illustration of a flow chart of a multi-pass process for a RAG system according to an embodiment of the invention.
(63) FIG. **62** is an illustration of a flow chart of a multi-pass process for a RAG system with caching and feedback loop according to an embodiment of the invention.
(64) FIG. **63** is an illustration of a mapper instance used in a multi-pass process for a RAG system according to an embodiment of the invention.
(65) FIG. **64** is an illustration of a reducer instance used in a multi-pass process for a RAG system according to an embodiment of the invention.
(66) FIG. **65** is an illustration of a probabilistic causal approach for a multi-pass RAG system, according to an embodiment of the invention.
(67) FIG. **66** is a flow chart illustrating how the Probabilistic Causal RAG system processes a legal analysis query, according to an embodiment of the invention.
(68) FIG. **67** is a flow chart illustrating how the Probabilistic Causal RAG system processes a stock market crash analysis, according to an embodiment of the invention.

DETAILED DESCRIPTION OF THE INVENTION

(69) The present invention will now be described more fully hereinafter with reference to the accompanying drawings, in which preferred embodiments of the invention are shown. This invention may, however, be embodied in many different forms and should not be construed as

limited to the embodiments set forth herein. Rather, these embodiments are provided so that this disclosure will be thorough and complete, and will fully convey the scope of the invention to those skilled in the art. Those of ordinary skill in the art realize that the following descriptions of the embodiments of the present invention are illustrative and are not intended to be limiting in any way. Other embodiments of the present invention will readily suggest themselves to such skilled people having the benefit of this disclosure. Like numbers refer to like elements throughout.

(70) Although the following detailed description contains many specifics for the purposes of illustration, anyone of ordinary skill in the art will appreciate that many variations and alterations to the following details are within the scope of the invention. Accordingly, the following embodiments of the invention are set forth without any loss of generality to, and without imposing limitations upon, the claimed invention.

(71) In this detailed description of the present invention, a person skilled in the art should note that directional terms, such as "above," "below," "upper," "lower," and other like terms are used for the convenience of the reader in reference to the drawings. Also, a person skilled in the art should notice this description may contain other terminology to convey position, orientation, and direction without departing from the principles of the present invention.

(72) Furthermore, in this detailed description, a person skilled in the art should note that quantitative qualifying terms such as "generally," "substantially," "mostly," and other terms are used, in general, to mean that the referred to object, characteristic, or quality constitutes a majority of the subject of the reference. The meaning of any of these terms is dependent upon the context within which it is used, and the meaning may be expressly modified.

(73) Referring now to FIG. **1** is an illustration of the training process for creating multiple specialized large language models for specific tasks/categories, is described in more detail. Data **100** (such as text, images, and audio) is used to pre-train a model in a process called unsupervised pre-training **102** which generates a base h-LLM model **104**. The pre-training process is referred to as unsupervised as unlabeled data is used at this step. The base h-LLM model **104** is then fine-tuned in a process called supervised fine-tuning **106**. The fine-tuning process uses smaller labeled data sets. The base h-LLM model **104** is fine-tuned to generate multiple h-LLM models which are specialized to perform specific tasks such as Question Answering, Information Extraction, Sentiment Analysis, Image Captioning, Object Recognition, Instruction Following, Classification, Inferencing, and Sentence Similarity, for instance.

(74) Referring now to FIG. **2** is an illustration of h-LLMs trained with different training sets, is described in more detail. As used in this specification h-LLM usually refers to a family of LLMs, such as those used in Google's Bard or OpenAI's GPT-4, that have been trained on a particular training set T. Therefore, the same family of LLMs (e.g., GPT) if trained on a different training set, T**1**, as opposed to GPT trained on training set T**2** could be differentiated as a separate h-LLM). The training sets can be private within an organization or public datasets.

(75) For example, as shown in FIG. **2**, h-LLM-**1 152** is trained with training set-**1 150**, h-LLM-**2 156** is trained with training set-**2 154**, h-LLM-**3 160** is trained with training set-**3 158**, and h-LLM-**3_4 164** is trained with training set-**3 158** and training set-**4 162**.

(76) An h-LLM can be described as a combination of LLM families and the training dataset used as follows:

h-LLM=LLM family (*X*) trained with Training Set (*Y*)

(77) For example, h-LLM_**1**=PaLM-**2** may be trained with training set T**12** h-LLM_**2**=PaLM-**2** may be trained with training set T**12**+T**45** h-LLM_**3**=GPT-4 may be trained with Training Set T**65** h-LLM_**4**=GPT-4 may be trained with ANY data set

(78) Referring now to FIG. **3**, an illustration of the process for generating synthetic data from multiple h-LLMs and using it for model refinement, is described in more detail. Data **200** is used to train a base h-LLM model **204** using unsupervised pre-training **202** which is then fine-tuned in a supervised fine-tuning process **206** to generate multiple h-LLMs specialized for specific tasks or

categories **208**. Each of these h-LLMs **208** are used to generate synthetic data **210** which is then fed back to the models in feedback loop **212** through a process called model refinement **214**.

(79) Referring now to FIG. **4** is an illustration of a bagging approach, that has some similarity to what was originally used in the context of machine learning models in a different way (for analytics as opposed to generative AI applications, such as LLMs) that are described in this invention, where multiple h-LLMs with lower precision and accuracy are merged/fused to create a merged h-LLM with higher precision and accuracy, is described in more detail. Bagging is a machine learning technique which improves the stability and accuracy of machine learning models. Using the input data **300**, multiple subsets of the data are created which are used to train multiple h-LLMs (**302**, **304**, **306**, **308**) in parallel. These models are then combined in a process called merging or fusing **310** to create a merged h-LLM **312**.

(80) Referring now to FIG. **5** is an illustration a boosting approach, that has some similarities to that originally used in the context of machine learning models in a different way (for analytics as opposed to generative AI applications used in this invention) where multiple h-LLMs of increasing precision and accuracy are created in a sequential manner and then merged/fused to create a merged h-LLM, is described in more detail. Boosting is a machine learning technique that involves creating a stronger and more accurate model from a number of weaker models. The original data **400** is used to train an h-LLM **402**. The h-LLM **402** is tested and the output **404** is assigned weights to generate weighted data **406**. The weighted data **406** is then used to train h-LLM **408**. The same process is then repeated and h-LLMs **414** and **420** are generated in a sequence. The h-LLMs **402**, **408**, **414** and **420** are then combined in a process called merging or fusing **424** to create a merged h-LLM **426**.

(81) Referring now to FIG. **6** is an illustration of creating a smaller and more specialized h-LLM through extraction/specialization process from a larger h-LLM, is described in more detail. The extraction/specialization process **502** extracts the specific knowledge required for a task from a big, general-purpose model, and creates a smaller h-LLM **506**. For example, a specific task can be sentiment analysis of input text, for which a smaller model **506** is more efficient as compared to a large, general-purpose model.

(82) Referring now to FIG. **7** is an illustration of combining h-LLMs trained with text, image and audio data to create a merged h-LLM, is described in more detail. Text data **600** is used to train h-LLM **602**, image data **604** is used to train h-LLM **606** and audio data **608** is used to train h-LLM **610**. The h-LLMs **602**, **604**, **608** are combined in a process called merging/fusing to create a merged h-LLM **614**.

(83) Referring now to FIG. **8** is an exemplary illustration of an application of using AI models for detecting labels in PDF files, is described in more detail. Patent documents (such as PDF files) have figures in which various entities/blocks/items are labeled using numeric labels (for instance **110**, **120** and so on). These labels are referenced and described in the patent text specification. When reviewing multiple documents, readers find it difficult to quickly lookup the labels mentioned in the figures (and what they refer to) from the text, as they need to go back and forth between a figure and the text in the specification. A novel PDF Label search solution is offered within CatchUp which allows quick lookup of labels in a figure using an innovative “AI Magnifier” approach. The user can select one or more labels using the Magnifier tool in the CatchUp GlassViewer (a PDF viewer tool within CatchUp that has annotation and other AI features). When one or more labels are selected using the Magnifier tool, the labels are searched within the PDF and the search results are returned. The PDF Label Search tool is built upon a novel AI Magnifier technology (which we refer to as AEye). AEye serves as a gateway to the world of Artificial Intelligence (AI) for documents and web pages. AEye can be used for a wide range of applications such as detecting objects in images, labels in documents, for instance. Documents or web pages **700** can be searched using an AEye application **704** which detects objects or labels utilizing an AEye backend **708**.

(84) Referring now to FIG. **9** is an illustration of generating derived prompts for different categories and using them with multiple h-LLMs to generate the best results, is described in more detail. User **800** enters a prompt in user interface **802**. The prompt is sent to the AI Input Broker **810** which generates multiple derived prompts for different categories. The derived prompts **822** are sent multiple h-LLMs **824** which produce the results. The results **816** are sent to the AI Output Broker **814** which processes the results and performs tasks such as filtering, ranking, weighting, assigning priorities, and then sends the best results to the user **800**. The h-LLMs **824** can have varying levels of accuracy, and optimized for different tasks such as Question Answering, Information Extraction, Sentiment Analysis, Image Captioning, Object Recognition, Instruction Following, Classification, Inferencing, and Sentence Similarity, for instance. The AI Output Broker **814** computes various scores and assigns weights for ranking the results. The results may be sent back to the h-LLMs till a certain level of accuracy or service level assurance is reached. The AI Input Broker **810** and Output Broker **814** update a local AI Broker Database **820** with the results of the request's path through its hierarchy and create an index of “derived requests” that may be used in future to select which set of “derived requests” an incoming request may fall into for further processing.
(85) Referring now to FIG. **10** is an illustration of using multiple h-LLMs to answer questions from specific input documents, is described in more detail. User **900** enters a prompt in user interface **902**. The prompt is sent to AI Input Broker **810** which generates multiple derived prompts for different categories **924**. The prompts are converted into embeddings using multiple embedding models **926**. The prompt embeddings **928** are sent to a vector database **930** which returns a list of knowledge documents **934** that are relevant to the prompt based on the similarity of their embeddings to the user's prompt. The knowledge documents **934** are sent to the AI Input Broker **810** which creates new context-aware prompts based on the user's initial prompt **916**, derived prompts **924** and the retrieved knowledge documents **934** as context and sends it to multiple h-LLMs **912**. The results produced by multiple h-LLMs are processed by the AI Output Broker **908** and the best result is sent to the user **900** along with citations from the knowledge documents **934**.
(86) Referring now to FIG. **11** is an illustration of an AI Broker for processing results from multiple h-LLMs, is described in more detail. Results produced by multiple h-LLMs **1000** are sent to an AI Output Broker **1002** which performs tasks such as assigning priorities **1004** and weights **1006** to the results, filtering **1010**, ranking **1012** and caching **1014**. The AI Output Broker **1002** provides an API interface **1016** for configuring and managing various aspects of the broker. An AI Broker Database **1020** stores the results along with the meta-data information such as the request path. AI Broker Database **1020** creates an index of “derived requests” that may be used in future to select which set of “derived requests” an incoming request may fall into for further processing.
(87) Referring now to FIG. **12** is an illustration of the combining h-LLMs in series, is described in more detail. User **1100** enters a prompt in user interface **1102**. The prompt **1104** is sent to an AI Input Broker **1106** which generates a derived prompt by adding more contextual information. The derived prompt is sent to multiple h-LLMs **1108** connected in series. The derived prompt goes to the first h-LLM in the sequence which generates results. The results of the first h-LLM are sent to the second h-LLM in the sequence for refinement/enhancement and then to the third h-LLM and so on. The AI Output Broker **1110** processes the results **1112** and sends the processed results to user **1200**.
(88) Referring now to FIG. **13** is an illustration of combining h-LLMs in parallel, is described in more detail. User **1200** enters a prompt in user interface **1202**. The prompt **1204** is sent to an AI Input Broker **1206** which generates multiple derived prompts by adding more contextual information. The derived prompts are sent to multiple h-LLMs **1208** which process the prompt in parallel generating multiple results. The AI Output Broker **1210** processes the results and sends the processed results **1212** to the user **1200**.
(89) Referring now to FIG. **14** is an illustration of a hybrid approach of combining h-LLM in series

and parallel, is described in more detail. User **1300** enters a prompt in user interface **1302**. The prompt **1304** is sent to an AI Input Broker **1306** which generates multiple derived prompts by adding more contextual information. The derived prompts are sent to multiple h-LLMs **1308** which processes the prompts generating one or more results. The AI Output Broker **1310** processes the results and sends the processed results **1312** to the user **1300**.

(90) Referring now to FIG. **15** is an illustration of the lambda architecture for h-LLMs, is described in more detail. Lambda architecture is a way of processing massive quantities of data that provides access to batch-processing and stream-processing methods with a hybrid approach, often utilizing in-memory storage instead of disks for speedier processing. Such in-memory processing may be accomplished using a volatile memory device such as random-access memory (RAM) devices, static random-access memory (SRAM) devices, dynamics random-access memory (DRAM) devices, magnetoresistive random-access memory (MRAM) devices, and the like, or a non-volatile random-access memory (NVRAM) device. Such processing may be done partially or entirely in-memory.

(91) This figure illustrates a lambda architecture for h-LLMs comprising batch layer **1402**, real-time layer **1404** and a query layer **1406**. New input data **1400** comes in continuously and is fed to the batch layer **1402** and real-time layer **1404** simultaneously. The batch layer **1402** maintains one or more h-LLMs which are updated/fine-tuned with the new data on a fixed schedule. Data is aggregated from the new input data **1400** over an aggregation duration that is tied to the fixed schedule. The real-time layer **1404** deals only with recent data which is not processed in the batch layer. The real-time layer **1404** maintains and updates smaller h-LLMs with incremental updates. The real-time layer **1404**, also utilizes Map Reduce type analytics and computing and processing (See for example, tutorialspoint.com/map_reduce/map_reduce_introduction.htm) of tokens in the tokenization processes to improve speeds by which tokens are merged or otherwise aggregated in a distributed GPU computing environment, User **1412** sends a prompt **1408** through user interface **1410** to the query layer **1406**. The query layer **1406** forwards the original prompt or creates one or more derived prompts which are sent to the batch and real-time layers. The query layer receives the results from the batch and real-time layers and performs tasks such as combining, ranking, filtering, assigning weights and priorities to the results and sends the best results to the user.

(92) Referring now to FIG. **16** is an illustration of batch and real-time processing architecture for h-LLMs, is described in more detail. The input data stream **1500** is sent to batch layer **1506** and real-time layer **1526**. The batch layer **1506** maintains a base h-LLM **1502** which is fine tuned **1504** in batch to generate fine-tuned h-LLM **1508**. The real-time layer **1526** generates smaller h-LLMs with incremental updates **1514** in real-time increments **1512**. The merger block **1516** combines and merges the h-LLMs from the batch layer and real-time layer to produce a combined h-LLM. The merged h-LLM is used with the query layer **1518** to respond to prompts **1520** sent by user **1524** through the user interface **1522**.

(93) Referring now to FIG. **17**, an illustration of an in-memory processing architecture for h-LLMs, is described in more detail. The input data stream **1600** is sent to the data receiver **1602** which breaks the data into small batches **1604** which can be processed at least partially, and in some embodiments entirely, in-memory. The processing layer **1606** includes multiple h-LLMs which process the batches on input data and produce the batches of processed data **1608**. Such batches may be produced after aggregating data from the input data stream **1600** over an aggregation duration.

(94) Referring now to FIG. **18** is an illustration of the architecture of PDF label search tool with CatchUp GlassViewer, is described in more detail. User **1700** uploads a PDF document **1702** to the CatchUp document management system **1704**. The text of the PDF document is extracted and indexed **1714** in the AEye backend system **1716**. Such extraction and indexing may be performed using character recognition analysis, including optical character recognition analysis. The user opens the PDF document **1706** with the CatchUp GlassViewer application **1708** in a browser. User

1700 launches the label search tool **1710** within the CatchUp GlassViewer application **1708** and selects a label using the magnifier tool. The selected label is sent to the AEye backend system **1716** which retrieves and returns **1718** all occurrences of the label.

(95) Referring now to FIG. **19** is an exemplary interface **1800** of the CatchUp platform showing the document management system, is described in more detail. Within this interface users can create new documents, upload existing documents, view and edit the documents.

(96) Referring now to FIG. **20** is an exemplary interface **1900** of the CatchUp platform showing the PDF viewer (GlassViewer), is described in more detail. GlassViewer is a PDF viewer application with CatchUp that allows annotating and commenting PDF files. The annotations and comments are stored in a separate layer which is rendered above the PDF document.

(97) Referring now to FIG. **21** is an exemplary interface **2000** of the CatchUp platform showing a magnifier tool **2002** within the GlassViewer for searching labels, is described in more detail. GlassViewer includes a PDF label searching tool called AEye Label Searcher that allows quickly searching for all occurrences of selected labels within the PDF. AEye Label Searcher uses a magnifier to select specific labels within a region of the PDF which are sent to the AEye backend for processing, and the results are then displayed, which include excerpts from the document where the labels are mentioned. In some embodiments, the AEye backend may lookup labels within multiple documents or return additional information generated from one or more h-LLM models as taught elsewhere in other embodiments of this invention. For example, a legal brief may be first generated using a local (in-house) database of briefs and then supplemented by h-LLMs that are trained on public-domain training sets of legal briefs, and the combination may be merged as needed.

(98) Referring now to FIG. **22** is an exemplary interface of the CatchUp platform showing label search results within GlassViewer, is described in more detail. The labels selected using the magnifier within the AEye Label Searcher are sent to the AEye backend for processing and the results are then displayed as shown in this figure.

(99) Referring now to FIG. **23**, an illustration of the process used by organizations to decide the list of tasks for a project, is described in more detail. Given an objective **2200** such as applying for a business loan, the manager and team **2202** in an organization use their experience or current company process documents to come up with a plan in an ad-hoc manner. The plan includes a list of things to do **2204**. The list of things to do may be informed by procedures and guidelines **2208** that may be provided by an administrative or governing body, such as, for example, the Small Business administration, and may be retrieved from a website **2210** therefor.

(100) Referring now to FIG. **24**, an illustration of the process of training LLMs to generate project, tasks and action items, is described in more detail. A Corpus of Company-Specific Processes and Procedures and Documents **2300** and Corpus of Public Processes and Procedures and Documents **2302** are used as input data for training of LLMs to generate Project, Tasks and Action Items **2304**. The content generated by the LLMs may be designated as supervised or labeled learning data **2308**. During the training process inputs such as requirements, constraints, assumptions, dependencies and risks **2310** are given. The trained LLMs are used to generate synthetic data which is then fed back to the models in feedback loop through a process called model refinement **2306**.

(101) Referring now to FIG. **25**, an illustration of the process of creating a project within CatchUp using a prompt, is described in more detail. Within CatchUp, the user enters a prompt **2400** (such as create a project for implementing a pre-sales process in an IT Company). The prompt **2400** is sent to the AI Input Broker **2402** which generates one or more derived prompts **2410**. The derived prompts **2410** are sent to one or more h-LLMs **2404**, preferably several h-LLMs, which produce the results. The results **2412** are sent to the AI Output Broker **2406** which processes the results and performs tasks such as filtering, ranking, weighting, assigning priorities, and then sends the best results to the user. CatchUp **2408** creates a project and list of tasks and sub-tasks/action items within the project based on the response received from the AI Output Broker.

(102) Referring now to FIG. **26**, an illustration of the process of Retrieval Augmented Generation (RAG), is described in more detail. RAG is a paradigm for augmenting LLM with custom data. A knowledge base **2500** is prepared from a corpus of documents (such as company-specific documents or public documents) in an indexing stage. The knowledge base **2500** can then be queried in the querying stage in which the relevant context **2502** is retrieved from the knowledge base to assist the LLM **2506** in generating a response **2508** to a query **2504**.
(103) Referring now to FIG. **27**, an illustration of the process of indexing documents for RAG, is described in more detail. The data source **2600** comprises documents (such as company-specific documents or public documents). The data loader **2602** reads the data source files and parses them to create the parsed documents **2604**. The parsed documents **2604** are then indexed in an indexing stage **2606** in which the documents are converted into vector embeddings, and related metadata is inferred. The vector embeddings of documents are stored in a vector database which serves as the knowledge base **2608** for RAG.
(104) Referring now to FIG. **28**, an illustration of the process of querying documents in RAG, is described in more detail. In the querying stage, the most relevant documents from the knowledge base **2700** are retrieved as the relevant context which is passed to the LLM along with the user's query. The relevant context allows the LLM to respond to the user query (including through use of the derived queries) even though the LLM may not be trained with the data from the knowledge base. The retrievers **2702** are used to retrieve the relevant context from the knowledge base **2700**. The retrievers **2702** may use different search and similarity matching strategies to search and retrieve the documents. Post-retrieval, the post-processor **2704** filters, transforms and ranks the relevant documents. The response synthesizer **2706** generates a response from an LLM, using a user query. The query engine **2708** allows users to ask questions over their own data. The chat engine **2710** allows users to have chat conversations with their data. The agent **2712** is an automated decision maker that interacts with external tools.
(105) The knowledge base contains a large set of documents (which can be private documents that are used to generate a “context” for the trained LLM″. The Retriever may use efficient search algorithms, such as keyword-based or page-rank based searches for a cluster or subset, to identify the relevant context that is then used by the Retriever to generate the context that is they sent to the Query engine, or Chat Engine or LLM Agent. This is a two-step process that adds to how existing RAGs operate and removes the limitation that current LLMs start with an unsuitable context, and thus cannot do better. The same approach can also be used when training the LLM initially or fine-tuning it. Our approach thus combines traditional search algorithms of a set of documents to find the “best match” through traditional search mechanisms (e.g., page rank, or cluster search algorithms (both online, internet, and offline or local modes) to identify the suitable (e.g., based on a matching score of the subset of relevant documents or their portions/blocks/chunks) cluster, which may be ordered or reordered or transformed relevant to the prompt/query (or the derived queries and/or prompts) and associated context.
(106) Referring now to FIG. **29**, an illustration of the RAG pipeline is described in more detail. RAG is a paradigm that allows combining LLMs with custom data. The RAG pipeline includes indexing and querying stages. In the indexing stage, documents and custom data **2800** (such as such as company-specific documents or public documents) are loaded and parsed. Next, the documents **2800** are split into smaller chunks using text splitters creating document splits **2804**. The splits **2804** are then stored in a vector database **2806**, such as ChromaDB, along with extracted metadata. Relationships between the document splits (or chunks, contexts, or blocks) may be stored in a graph database, such as Neo4j or GraphRAG (that was open-sourced by Microsoft in **2024**). The documents **2800** are stored in the form of vector embeddings in the vector database which allows fast searching, retrieval and comparison of documents using different similarity measures and/or through use of the metadata. This completes the indexing stage. In the querying stage, the user sends a query **2802**. The system retrieves splits from the vector database which are similar to

the query. These retrieval splits **2818** and relevant associated metadata and processing results are then sent to the LLM as the context information along with the query as a prompt **2814**. The prompt **2814** comprises the user query **2802** and the retrieval splits **2818**. The LLM **2816** generates the answer **2812** based on the query **2802** and context information in the prompt **2814**.

(107) Referring now to FIG. **30**, an illustration of an approach that uses multiple RAG pipelines with clusters of documents, is described in more detail. Existing LLMs and RAG pipelines have low accuracy and increased memory and performance penalties because of the way they develop context in normal high-level API use. A problem with LLMs includes the context being typically small (e.g. 4 k tokens) as opposed to much larger context size, that could be 100 k tokens or more. The context is not relevant and is created or selected through techniques that may not create the highest relevance score. To address these limitations an approach of using multiple retrieval augmented generation pipelines with clusters of documents is proposed. Based on a set of user queries **2900** the documents **2902** are searched **2904** and clustered **2906**. The proposed approach thus combines traditional search of a set of documents to find the "best match" through traditional search mechanisms (e.g., page-rank), or cluster search algorithms (both online and offline modes) to identify the suitable (e.g., based on a matching Score of the subset of relevant documents) cluster and associated metadata relevant to the query and its context. For each document cluster **2906**, a separate RAG pipeline is created **2916**, **2918**, **2920** and produces its own result **2910**, **2912**, **2914**. The user queries are then routed to the most suitable RAG pipelines **2908** using a similarity/matching score and/or processing based on relationships between documents and/or their chunks and each other across the entire document set (utilizing, for example, a graph database as well for processing of relationships and/or metadata).

(108) Referring now to FIG. **31**, an illustration of an approach that uses a prompt augmented with a search query, is described in more detail. Currently, in RAG pipelines, the prompt is used by both the Retriever and also the LLM. The language in the prompt may cause the retriever to generate low quality context that is then sent to the LLM, and results in poor quality results from the LLM. Instead, the prompt that we propose can be parsed or filtered or processed first to create a search query that is used to retrieve a subset of documents and/or their portions that have the best match with the search query derived from the prompt.

(109) Prompt **3004** has a portion that is the prompt for the LLM **3006**, and a portion that is a search query **3002** for creating a subset of the documents to be used by the retriever for its matching function. The search query **3002** portion of the prompt is used to search a large corpus of documents **3000** and create small document clusters (such as corpus **3008** and **3010**). For each document cluster, a separate RAG pipeline may be used (such as **3020** and **3022**).

(110) The prompt **3004** is processed to separate out a search query (before the documents are indexed into the vector database, and the full corpus of documents is broken (possible offline) into smaller clusters or chunks of documents. Based on the augmented prompt (that contains the prompt and search query), one or more clusters **3008**, **3010** are chosen to answer the query and the corresponding RAG pipelines **3020**, **3022** are then used to generate responses **3016**, **3018** to the prompts **3012**, **3014**. For example, consider a document corpus of a large number of PDF files related to real-estate contracts. The prompt is processed to separate out a search query and the full corpus of PDF files is broken into a rental contract subset/cluster and a purchase contract subset/cluster. Based on the query, the rental contract template subset/cluster is chosen and then the RAG pipeline processes the query in the context of rental contracts or their portions.

(111) The processing of the prompt can be done in two ways—the prompt specifically and explicitly identifies "rental option" as a separate field or word, or the "rental option" may be derived from the prompt because the prompt uses "without a downpayment". The context that results, e.g., "Rental Contracts", is more accurate and responsive to the prompt that a combined context that is generated from all the PDFs in the original collection. This partitioning into subsets can be done offline, and the "search portion" of the prompt can be further processed by mapping or

filtering or inference to the closest of the subsets of the documents in local or online databases, in case online creation of the subsets takes too long. Sometimes, online creation of the subsets of the collection of PDF files (or files or objects of other kinds, like images, videos, or text or songs) may be possible. In all cases accuracy is improved because the retriever is operating on a more relevant subset of documents or their portions based on their content and/or associated metadata.

(112) Referring now to FIG. **32**, an illustration of an approach that uses a search query to extract relevant context for the RAG pipelines, is described in more detail. A search query **3102** is used to extract documents **3108** from a large corpus **3100** and create smaller clusters of documents **3110** and **3112**. The search query **3102** is sent to an AI Input Broker **3104** which generates multiple derived queries which help in clustering documents into different clusters **3110** and **3112**. For each document cluster **3110**, **3112**, a separate RAG pipeline **3114**, **3116** is created. The prompt **3118** is sent **3120** to each of the RAG pipelines **3114**, **3116** and the responses from these pipelines are combined by the AI Output Broker **3122** which processes the results and performs tasks such as filtering, ranking, weighting, assigning priorities, and then sends the best results (in batch and/or real-time modes) **3124** to the user utilizing suitable APIs and supported by cloud-based container or virtualized systems, such as Kubernetes and Istio Service Mesh.

(113) Referring now to FIG. **33**, an exemplary interface of the CatchUp platform for RAG, is described in more detail. The exemplary interface **3200** is designed for legal use cases and allows users to query public and private legal documents obtained from courts and other sources. Different query modes **3204** are supported for data sources such as public, private and company specific document clusters, online and offline. For querying documents, different query types **3206** are supported such as legal research, legal citation, legal case brief, legal writing and legal argument. User can optionally provide attributes **3202** such as legal issue, represented party, jurisdiction, case name, court name, year, statute name, area of low, for instance. The user query **3208** is sent to the CatchUp backend server which implements multiple RAG pipelines as shown in FIGS. **31** and **32**.

(114) Referring now to FIG. **34**, an exemplary interface **3250** of the CatchUp platform showing response from Retrieval Augmented Generation (RAG) pipeline, is described in more detail. In response to a user query **3252** sent from the user interface **3250**, as shown in FIG. **33**, the response **3254** is returned as shown in FIG. **34**. The response **3254** includes the links to the source documents (or their portions or chunks, thereof) **3256** which were used as the relevant context with the RAG pipeline for generation of response.

(115) Referring now to FIG. **35**A, an illustration of using a single LLM as known in the prior art is depicted. Currently a single large LLM **3302** is trained on all types of data and has large number of parameters (e.g. OpenAI GPT3.5 has 175 billion parameters and GPT-4 has over 1 trillion of parameters). The single large LLM **3302** responds individually to the prompt **3300** to generate the answer **3304**.

(116) FIG. **35**B presents an illustration of using a network of LLMS. In an embodiment of the present invention, smaller LLMs (with 3 billion or 7 billion parameters, for example), each focused on a specific type of result (cost estimation, profit estimation, expense estimation or prediction) are provided in a network **3308**, and then network of LLMs **3308** is used to provide a composite result that is easier to prompt for, easier to optimize and easier to “explain” how it works by having smaller focused LLMs trained on specialized training sets. The smaller LLMs can be chosen from a library of LLMs and mixed and matched with other compatible LLMs using a communication protocol that exchange messages and exchange messages with a head-end or Manager LLM using an architecture that could be similar to Hadoop (where there is a master manager and bunch of worker nodes). Here is there is a master LLM **3312** and several networked worker LLMs **3314**. An adaptor module or a set of adapters **3316** may be configured to process these inter-LLM messages and inter-h-LLM messages and serve as a router for converting the format of a particular LLM's output to fit to the format of the API for another LLM and then route it on a cloud-container based virtualized network, similar to Istio Service Mesh that is used for routing services in typical cloud-

based container implementations of computing microservices, that may be optionally based on GPUs. Therefore, if there are two choices available, one is to use an API Gateway that receives the APIs and converts and adapts the APIs for various LLMs and calls between LLMs, and another could be based on a service mesh that has agents, for example, that route the prompts and queries to the respective LLMs and process the results and redirect them to the appropriate destination. Other options are also available in batch or real-time modes.

(117) The LLMs in a network can communicate with each other using client-server protocols such as HTTP/HTTPS, FTP, RPC, or peer-to-peer network protocols such as BitTorrent protocol, IPFS, WebRTC, for instance. Custom protocols may be developed for efficient communication of LLMs in a network. An example use case can be a network of Vision LLMs (one for each car on a road or in a platoon) which exchange information as to unusual traffic incidents that the autonomous car may not have been trained with to handle.

(118) Referring now to FIG. **36**, a comparison of the current and proposed approaches with the LASER system and SCORE-RAG system is described in more detail. FIG. **36**A illustrates the prior art approach, FIG. **36**B illustrates the proposed approach for processing long documents with LASER, and FIG. **36**C illustrates the proposed approach for optimizing RAG with SCORE-RAG. In the prior art approach a User **3400** sends **3606** one or more long documents (as context) **3402** along with a prompt (containing the user's query) to an LLM **3406**. The LLM processes the prompt and the context and generates a response **3408**. The prior art approach is limited by the context length limit of the LLM, therefore, only a limited number of small documents or only a portion of a long document can be processed. Moreover, the LLM faces challenges when processing long documents, particularly in maintaining coherence and performing long-range reasoning due to the attention span problem which causes a noticeable drop in performance as the length of the input context increases, typically beyond 10,000 to 50,000 tokens.

(119) As shown in FIG. **36**B, an embodiment of the present invention, the LASER system **3414**, uses an iterative attention-focusing technique that dynamically refines and condenses document context to improve model comprehension and coherence over extended inputs. The process begins with a User **3410** who has one or more Long Documents **3412** for processing. Instead of sending the document directly to an LLM **3418**, long document(s) **3412** are first passed through a LASER system **3414**. The LASER system **3414** employs the iterative attention-focusing technique to refine and distill the context of the long document and its chunks. After refinement, the refined documents (or refined generated documents and subsets/chunks/blocks) **3416** (containing processed and condensed information and also extracted metadata, optionally stored in a graph database) are then sent to the LLM **3418** for final analysis or generation tasks. One distinction between the approaches in FIGS. **36**A and **36**B is the introduction of the LASER system **3414** for attention span refinement. The LASER system **3414** acts as an intermediary processor that prepares the long document(s) **3412** for more effective handling by the LLM **3418**. The LASER system **3414** addresses the limitations of the prior art solution by introducing an intelligent preprocessing step which enables LLMs to effectively handle long documents that would otherwise exceed their attention capacity.

(120) FIG. **36**C illustrates another embodiment of the invention, a SCORE-RAG system **3430** which can be combined with a LASER system **3426** to create a solution for processing long documents and generating accurate responses that may perform better than the prior art solution. The process begins with a User **3422** who has one or more long documents **3424** for processing. The documents and/or their chunks are processed through the LASER system **3426** which produces refined documents **3428** which are then fed to the SCORE-RAG system **3430**. The SCORE-RAG system **3430** uses context-optimized retrieval techniques to enhance the traditional RAG approach. This system enhances the traditional RAG approach by incorporating advanced techniques such as topic modeling, intelligent document chunking, citation analysis, and hybrid search methods. The SCORE-RAG system **3430** processes the refined documents **3438** to create optimized chunks of information (and associated metadata based relationship information between chunks themselves

(within a document and across documents) and also between chunks and documents), which are then indexed and prepared for efficient retrieval and use in language generation by the LLM **3432**. New documents or documentation may be assembled (including reordering) “on-the-fly” in batch mode or real-time modes based on this processing. At a high level, without restricting or limiting the invention, given a corpus of documents, the LASER system **3426** processes these documents, document to document, to look at portions of relevance and interest (not in response to a particular user query, usually) but based on categories or types of information that may be present in each document that may be of interest. A long document processed by the LASER system **3426** results in a refined document. The processing of long documents by the laser system **3426** is usually perform, document by document, offline. The refined document may also be a summary, or a subset, or portions (or chunks) of a document that are of most interest in general, by a category. The LASER system **3426** may itself utilize an LLM to help it identify portions or chunks of most relevance to a category of information in the process of creation of the refined documents. The refined documents **3428** may then be taken by the SCORE-RAG system **3430** which may, in response to a query, look at the refined documents **3428** and their tagged portions or chunks organized along with metadata and relationships to each other and to categories, for example, and carry out operations on ranking and scoring those chunks that are most relevant to a particular query or a set of queries, for example, a batch of queries or derived queries.

(121) The LASER system **3426** usually performs offline pre-processing that is not directly related to user query or derived query. The LASER system **3426** processes all documents one by one and identifies portions (or chunks or contexts) of them that fall into one or more categories of interesting information, for example, receipts, bills, tax data, customer names, inventories, etc. The LASER system **3426** then redefines each document to call out these portions and adds meta-data to identify the categories. Additionally, the LASER system **3426** may summarize the portions/chunks/contexts. Multiple categories can exist for the same portion, for example, a single portion can be in the receipt and tax categories. The LASER system **3426** may then repeat this process for all long documents. It can use a LLM or other search means (e.g, cluster search) for this process. This entire process can be done once or relatively few times for each document/Typically, the LASER system **3426** performs this process not responsive to a real-time query or prompt from the user. It can be done each time a long document is loaded into the system. The prompts from users can be used to create categories that are used in the LASER system **3426**.

(122) The SCORE-RAG system **3430** may perform online real-time processing to create relevant context in response to a derived prompt or a user prompt. The SCORE-RAG system **3430** may then take the refined documents **3428**, which are categorized chunks from these documents and other meta-data, including relationships between chunks and documents, and then performs scoring, ranking, collection, and collation in response to a user query and/or its derived queries. The highly scored chunks may then be sent to the LLM **3432** for processing to create results for the user. The SCORE-RAG system **3430** can use graph and vector databases to create chunks that are most relevant to the user prompt of derived prompts and send the context to the LLM.

(123) The combined approach using the LASER system **3426** and the SCORE-RAG system **3430** may provide improvements over prior solutions. By first refining the document through the LASER system **3426**, the subsequent RAG optimization process can work with a more focused and coherent input, potentially improving its effectiveness. The refined documents **3428** (or refined generated documents and subsets/portions/chunks/blocks) allows for more accurate topic modeling and chunking in the SCORE-RAG system **3430**, leading to better retrieval results. The iterative refinement process helps maintain important contextual information, which can then be effectively utilized in the RAG system's retrieval and generation stages. Both the LASER system **3426** and the SCORE-RAG system **3430** are designed to handle large volumes of text, making this combined approach suitable for processing extensive document collections or extremely long individual documents. This integrated system can be applied to a wide range of document types and domains,

such as scientific literature, legal documents, and technical manuals, for instance.
(124) Referring now to FIG. **37**, an illustration of the high-level architecture of a LASER system **3500** according to an embodiment of the invention is described in more detail. The LASER system **3500** architecture comprises five primary modules, including an input module **3502**, a model module **3504**, an iteration controller **3506**, a knowledge module **3508**, and an output handler **3510**. Each of these modules contains sub-components that perform specific functions within the overall system. The input module **3502** serves as the entry point for document processing. It comprises a long document parser, which is operable to ingest long documents and breaking them down into manageable blocks, and an iterative context generator, which is operable to create and refine batches of blocks from the long document processor for processing in subsequent stages. The model module **3504** forms the core processing unit of the system. It comprises, and in some embodiments consists of, an attention model which is an LLM that is operable to process and comprehend the document content, and a ranking Unit that is operable to evaluate and score the outputs produced by the attention model, identifying the most relevant and coherent content. The iteration controller **3506** is operable to control the flow of the iterative process. It comprises a loop control, which is operable to determine when to continue or terminate the iteration process based on predefined criteria, and an output aggregator, which is operable to collect and consolidate the results from multiple iterations. The knowledge module **3508** is operable to enhance the ability of the LASER system **3500** to distill and organize information. The knowledge module **3508** comprises an extractive summarizer, which is operable to identiy and extract key sentences or passages from the processed content, a document clustering component, which is operable to group related content/blocks/chunks based on semantic similarity or relevance to a topic, and a virtual document creator which assembles chunks (in any order or sequence or through creation of summaries) together to create an improved document on a topic relative to the long document(s). The output handler **3510** is operable to prepare the processed information for presentation. The output handler **3510** comprises a response filter, which is operable to perform a final validation of the output to ensure quality and relevance, and a result presentation component, which is operable to format the output for user consumption, potentially including visualizations or explanations. This architecture enables the LASER system **3500** to progressively refine and focus attention on the most relevant parts of long documents through multiple iterations.
(125) Referring now to FIG. **38**, a flow chart of the LASER system depicting a process by which long documents are processed and refined to enhance the attention span of LLMs according to an embodiment of the invention is described in more detail. The process begins with a long document **3600**, which serves as the input to the LASER system. It is contemplated that multiple long documents may be provided. At step **3602**, the document is split into smaller, manageable blocks/chunks. The block/chunk size is configurable, typically ranging between 10,000 to 50,000 tokens. This step facilitates breaking down the long document into sizes that are more aligned with the typical attention span of current LLMs. Following the splitting operation, the blocks/chunks undergo a shuffling process at step **3604** to mitigate any potential position bias that might be present in the original document structure. By randomizing the order of the blocks/chunks, the system ensures a more uniform treatment of the content during subsequent processing stages. Alternatively, reordering of blocks/chunks may be done such that most similar blocks/chunks are either at the beginning or at the end, to avoid attention span problems with LLMs. The shuffled/reordered blocks/chunks are then organized into initial batches at step **3606**. These batches serve as the first set of inputs for the LLM processing stage. The batching strategy allows for efficient processing while maintaining a balance between context size and manageability. An iterative process **3608** begins at step **3610**, where the batches are processed through an LLM. The processing step **3610** involves applying the capabilities of the LLM to comprehend and reason over the content of each batch. The outputs generated by the LLM are then subjected to a ranking process at step **3612**. This step evaluates the quality of the LLM's outputs, typically based on

factors such as coherence, relevance, and alignment with the original document's intent. At step **3614**, the system determines whether the iteration process should continue or conclude. This decision is based on predefined criteria, which may include factors such as the number of iterations completed, the degree of content refinement achieved, or specific quality thresholds being met. If the decision is to continue the iteration, the process moves to step **3618**, where the top K of the ranked blocks/chunks are selected, where K is a configurable parameter, typically between 10 to 50%. This selection process ensures that only the most relevant and coherent content is carried forward to the next iteration. The selected blocks/chunks then undergo a clustering operation at step **3618**. This step groups related content together, further refining the organization of the information based on semantic similarities. At step **3622**, new batches are formed from these clustered blocks, creating a more focused and refined set of inputs for the next iteration. These new batches are then fed back into the LLM processing stage **3610**, beginning the next iteration cycle. The iterative process **3608** continues until the termination criteria are met, at which point the process concludes, outputting the final condensed version of the document **3616**. This output represents a distilled version of the original long document, containing the most relevant and coherent information as determined by the iterative attention focusing process of the LASER system.

(126) Through this iterative approach, the system progressively refines and focuses the content of long documents, enabling LLMs to more effectively process and reason over extended contexts. This method addresses attention span limitations of current LLMs and provides a novel solution for handling long-form content across various domains. The LASER system is operable to iteratively refine the context to focus on the most relevant information to help the LLM maintain coherence and attention.

(127) Aspects of a LASER system according to an embodiment of the invention include splitting long inputs into multiple blocks/chunks (e.g. **10***k*-**50***k* tokens each) and processing the blocks/chunks separately, ranking and/or scoring the outputs from each block/chunk to determine most relevant content, constructing new condensed blocks/chunks from the highest scoring outputs for next round and/or delete irrelevant information, iteratively processing the blocks/chunks through an LLM, selecting the best outputs, and condensing or summarizing the best outputs, and gradually concentrating the context into fewer but more focused subsets of documents or chunks or summaries.

(128) The LASER system allows an LLM to digest manageable sized contexts, select useful results/signals, and carry those forward in a refined set of inputs. The LASER system works similar to a funnel by distilling the context. The ranking/selection steps of the LASER process improve the ability of an LLM to identify and focus on the pertinent content across rounds of processing. The LASER system guides the LLM's limited attention by incrementally removing irrelevant information and emphasizing the best results/signals from prior rounds. LASER system can have different variations, such as having multiple rounds of a large number of small blocks before condensing, however, the incremental refinement of context and attention through derived inputs remains the core idea.

(129) Referring now to FIG. **39**, an illustration of an adaptation of the MapReduce model for the LASER system, is described in more detail. The figure presents a flow chart depicting how the concepts of mapping, translating, shuffling, and reducing are applied to the iterative attention focusing approach of the LASER system, enabling efficient processing of long documents. The process begins with a long document **3700**, which is the input to the system. The long document **3700** is split into blocks/chunks at step **3702**. Step **3702** prepares the document for distributed processing. The system then enters the map phase at step **3704**, which is a component of the MapReduce paradigm. In the map phase, each block/chunk is independently processed by the LLM. This is represented by the parallel processes **3706**, **3708** and **3710**, each handling a separate block/chunk. The number of these parallel processes can be scaled based on the available computational resources, allowing for efficient handling of long documents. Following the map

phase, the system moves to the shuffle & sort stage at step **3712**. This stage facilitates reorganizing the processed information from the individual blocks/chunks. It allows for the redistribution of related content across the processed blocks, preparing for the subsequent reduce phase. The reduce phase begins at step **3714**, where the shuffled and sorted data is consolidated. The reduce phase is represented by processes **3716**, **3718** and **3720**, which perform ranking and clustering operations on batches of the processed and reorganized blocks/chunks. The number of reduce processes can be adjusted based on the desired granularity of the reduce operation. The results from the reduce phase are then combined at step **3722**, creating a consolidated view of the processed document. At this point, the system evaluates whether the iteration is complete at step **3724** based on predefined criteria such as the number of iterations, the degree of refinement achieved, or specific quality thresholds. If the iteration is not complete, the system forms new blocks/chunks at step **3728** based on the reduced and refined content. These new blocks/chunks are then fed back into the map phase **3704**, beginning another iteration cycle. This iterative loop allows for progressive refinement of the document's content, gradually focusing on the most relevant and coherent information. When the iteration criteria are met, the process concludes, outputting the final refined output at step **3726**. This output represents a distilled version of the original long document, containing the most relevant and coherent information.

(130) This adapted MapReduce model may be advantageous for the proposed attention span refinement system. The parallel nature of the map phase allows for efficient processing of extremely long documents by distributing the computational load. The number of map-and-reduce processes can be adjusted based on available resources and specific requirements of the task. The cyclical nature of the process, facilitated by the iteration check and feedback loop, allows for progressive improvement of the document's focus and coherence. This model enables the system to leverage distributed computing architectures, potentially improving processing speed for very large documents. Processing of documents can be done in a fault tolerant manner, allowing for robust processing of long documents even in the face of potential hardware or software failures. By adapting the MapReduce model to the attention span refinement process, the LASER system provides an efficient and scalable approach to handle long documents, and address the limitations of current LLMs.

(131) Referring now to FIG. **40**, an illustration of the architecture of a SCORE-RAG system according to an embodiment of the invention is described detail. The SCORE-RAG system **3800** comprises several modules, each designed to perform specific functions in the overall process. A document processor **3802** serves as the entry point for the system. It includes an input handler operable to receive documents in various formats, a text extractor operable to parse and prepare the text comprised by the received documents for further processing, for example, by creating chunks from the text, and a parent document tracker operable to maintain relationships between original documents and their chunks. A topic modeling engine **3804** is operable to analyze the received document to identify main topics and themes. It comprises, and in some embodiments consists of, a topic analyzer and a theme identifier, working in tandem to create a semantic map of the document. A document chunking module **3806** is operable to break down the received document into semantically meaningful segments. It includes a semantic segmenter for intelligent text division, a topic tagger operable to associate relevant topics with each chunk, and a multi-format generator operable to create multiple representations for each chunk (e.g., summaries, hypothetical questions related to the chunk). A citation analyzer **3808** comprises a reference extractor and an impact assessor. The citation analyzer is operable to identify and evaluate citations within the received document, contributing to the overall relevance scoring of different text segments. A chunk selection and ranking module **3810** employs a relevance scorer in being operable to assess the importance of each chunk, a top chunk selector in being operable to identify the most pertinent segments for indexing and retrieval, and a time-weighted scorer which is operable to incorporate the timestamps into chunk scoring (where more recent chunks may be ranked higher). A metadata

enrichment module **3812** is operable to enhance the selected chunks with meta-data. This module uses a tag assigner to append relevant metadata, a context enhancer to add additional contextual information, and a timestamp tracker to record creation and access times for time-weighted retrieval. An indexing engine **3814** comprises a vector database, a graph database and a full-text search engine. This module is operable to prepare the processed chunks for efficient storage and retrieval. Multiple vector representations of the same document or chunk are stored (e.g. summarized versions, tagged versions, hypothetical questions for document/chunk, etc). A query processor **3816** is operable to handle user inputs. This module includes a query analyzer for understanding query structure, an intent classifier for determining the user's information needs, and a derived-query/prompt generator which is operable to create multiple perspective queries/prompts from a user input. A retrieval orchestrator **3818** is operable to coordinate the retrieval process. This module includes a cache checker for rapid information access and a search dispatcher for initiating more complex searches when needed. The search dispatcher implements different search and retrieval techniques such as vector similarity search, maximal marginal relevance (MMR) search, keyword-based search, cluster search (which groups similar documents/chunks into clusters based on their similarity and the query is match with the most relevant clusters) and ensemble search (which combines results from multiple search methods, for example, a combination of vector similarity and keyword based search). As disclosed in U.S. Provisional Patent Application Ser. No. 63/463,913, which is incorporated by reference herein above, a label or a search query results in a search to a set of documents to obtain the most relevant set.

(132) An augmentation engine **3820** may be operable to employ a context integrator to combine retrieved information with the original query and a query enhancer to enhance the query based on this additional context. A generation module **3822** may be configured to interface with an LLM. This module includes an LLM interface for query processing and a response generator for producing coherent and relevant responses. An evaluation and fine-tuning module **3824** includes a performance analyzer which is operable to monitor system-wide metrics and evaluate the response quality, and a model fine-tuner which is operable to adjust the system parameters for optimization.

(133) Referring now to FIG. **41**, a flow chart of a process performed by a SCORE-RAG system according to an embodiment of the invention is described in more detail. The process begins with a document **3900**, which may be a refined output from the LASER system or any other textual input requiring processing. The document **3900** undergoes topic modeling at step **3902** to identify key topics and categories within the text. The document is segmented into semantically meaningful chunks at step **3904**, with each chunk tagged according to the identified topics. Citation Analysis is performed at step **3906** which involves analyzing citations or references within the document to identify the most influential or frequently cited sources. Based on the topic relevance and citation analysis, the most important chunks of the document are selected for further processing at step **3908**. The selected chunks are enriched with metadata at step **3910**, including topic tags, citation information, and other relevant attributes. At step **3912** the processed and tagged chunks are indexed in a vector/graph database and/or a full-text search engine (such as, for example, Elasticsearch or Solr), enabling efficient retrieval. Indexing can be done in a combination of databases (vector/graph) and full-text search engines to enable efficient hybrid search at a later stage. At step **3914**, the system waits for a query/prompt input. When User **3916** provides the query it triggers the retrieval process. The retrieval stage begins at step **3918**. Upon receiving a query, the system initiates the retrieval process to find relevant information. At step **3920**, a cache check is performed. The system first checks if the required information is available in the cache/in-memory. If there's a cache hit, i.e. the required information is available in-memory, the information is fetched directly from the cache at step **3922**. In case of a cache miss, i.e. the required information is not available in-memory, the system performs a hybrid search at step **3924** to retrieve relevant information. At augmentation stage is performed at step **3926** where the retrieved information, whether from cache or search, is used to augment the original query. A generation stage is

performed at step **3928** where the system generates a response using the augmented query. The generated response undergoes an evaluation process at step **3930** to assess its quality and relevance. The system determines if the generated response meets the required performance criteria at step **3932**. If satisfactory, a final response is produced at step **3934**. If unsatisfactory, the system initiates a fine-tuning process at step **3936**. In the fine-tuning process, the system can tweak the models and parameters used in the topic modeling, document chunking and tagging, citation analysis, chunk selection, meta-data tagging, indexing and hybrid search stages, to improve the overall performance. The system may then iterate steps **3902** through **3912** and **3918** through **3932** until the generated response is determined to b

(134) The SCORE-RAG system of FIG. **41** may be advantageous over previous solutions. Through topic modeling and intelligent chunking, the system can better understand and retrieve relevant information. The use of caching and hybrid search techniques may enhance retrieval speed and accuracy. The fine-tuning process may allow the system to continuously improve its performance based on evaluation results.

(135) In the context of the SCORE-RAG system, it should be noted that the RAG optimizations can occur before or after a document is stored/indexed in a Vector database.

(136) In the context of the present invention, it should be noted that the term “Document” is used in a broad and inclusive manner. A person skilled in the art should understand that “Document” may refer to, but is not limited to: a traditional text-based document in its entirety; a portion or chunk of a larger document along with extracted metadata; a block of text, regardless of its source; a summary or abstract of a document; a combination of multiple distinct documents; a combination of summaries or chunks from multiple documents; a context or set of contextual information, including metadata; any digital content that can be processed as text, including web pages, emails, or social media posts; structured or semi-structured data that can be converted into a textual format; and/or a collection of related information, regardless of its original format or source.

(137) Referring now to FIG. **42**, an illustration of using the SCORE-RAG system for multi-modal RAG is described in more detail. The SCORE-RAG system **4002** of the present embodiment is designed with advanced multimodal capabilities, enabling it to process and generate a wide array of content types, including text, audio, video, software code, and images. This system handles multi-modal inputs **4000**, including but not limited to text, image, video, audio, and code, making it versatile for diverse applications. To achieve improved performance across these varied modalities, the SCORE-RAG system **4002** employs multiple embedding models **4004**, **4006**, **4008**, **4010**, **4012** and specialized vector databases **4014**, **4016**, **4018**, **4020**, **4022** each fine-tuned for a specific content type such as text, image, video, audio, or code. This specialized approach ensures that the unique characteristics and nuances of each modality are accurately captured and indexed. For the generation phase, the SCORE-RAG system **4002** utilizes an ensemble of LLMs or h-LLMs **4024**, each specialized for different tasks. These may include models optimized for question-answering, code generation, image interpretation, audio transcription, and video analysis, among others. This multi-faceted approach allows CORE-RAG to process a wide range of input types and also generate appropriate and context-aware multi-modal outputs. A User **4026** sends a query **4028** (comprising multi-modal inputs) to the SCORE-RAG system **4002** and received a multi-modal response **4030**. The multimodal capability of the SCORE-RAG system **4002** enhances the system's utility across diverse fields such as education, research, software development, multimedia content creation, and data analysis, for instance.

(138) Referring now to FIG. **43**, an illustration of a process of using a LASER system and a SCORE-RAG system for multi-modal RAG according to an embodiment of the invention is described in more detail. The process starts with a multi-modal input (for example, text, image, audio, video, or code) being ingested by the LASER system **4102** which is then refined/processed/distilled/filtered. The processed or refined data/documents (text, image, video, audio, code) are then fed to the SCORE-RAG system **4104**. The SCORE-RAG system **4104** can

integrate information from various sources and modalities, providing comprehensive and accurate responses that can combine textual explanations, code snippets, audio interpretations, and visual analyses as needed. For generation tasks, the SCORE-RAG system **4104** utilizes an ensemble of LLMs or h-LLMs **4106**, each specialized for different tasks. A User **4108** sends a query **4110** (comprising multi-modal inputs) to the SCORE-RAG system **4104** and receives a multi-modal response **4112**. The output from the SCORE-RAG system **4104** can also be fed back to the LASER system **4102** for iterative processing/refinement.

(139) Referring now to FIG. **44**, an illustration of the different meanings of "Document" in the context of the present invention, is described in more detail. A person skilled in the art should understand that "Chunk" or "Block" may refer to, but is not limited to: traditional forms **4232**, such as entire text-based document **4234**, a portion/chunk of larger document **4238**, or a block of text from any source **4236**; data forms **4202**, such as structured/semi-structured data convertible to text **4204**, or related information collection **4206**; digital forms **4208**, such as social media posts **4218**, web pages **4216**, emails **4210**, images **4212**, and audio/video **4214**; contextual forms **4220**, such as context or contextual information set **4222**; and derived forms **4224**, such as a summary/abstract of a document **4226**, a combination of summaries/chunks **4228**, or a combination of multiple documents **4230**.

(140) The meaning of "document" in any specific instance is dependent upon the context within which it is used, and the meaning may be expressly modified within the description of particular embodiments. This broad definition is intended to encompass the various ways in which textual or informational content may be presented, processed, or manipulated within the scope of the present invention.

(141) Referring now to FIG. **45**, an illustration of the different meanings of "Chunk/Block" in the context of the present invention, is described in more detail. In the context of the present invention, it should be noted that the terms "chunk" and "clock" are used in a broad and inclusive manner. A person skilled in the art should understand that "chunk" or "block" may refer to, but is not limited to: document-based **4302**, such as a portion or segment of a larger document or text **4304**, a fragment of a document that maintains some level of context or meaning **4306**, a semantically coherent section of text **4308**, regardless of its size, a section of a document defined by structural elements **4310** (e.g., headers, chapters, sections); content-based **4318**, such as a unit of text **4324** defined by a specific number of tokens, words, sentences, or paragraphs, a piece of information extracted from a larger context **4320**; processing-based **4334**, such as a unit of data processed or analyzed in a single operation, a unit of information in any digital format that can be processed as a discrete entity **4340**, a unit of text used for processing, indexing, or retrieval purposes **4338**; or organizational **4326**, such as a logical division of content that may span across multiple physical documents **4328**, a section of text or data defined by temporal or sequential ordering **4332**, a portion of a document or dataset selected based on specific criteria or algorithms **4330**.

(142) The terms "chunk" and "block" may be used interchangeably or with distinct meanings depending on the specific context within the description of the invention. The precise definition, size, or characteristics of a "chunk" or "block" may vary based on the particular embodiment, implementation, or application of the invention being described.

(143) Referring now to FIG. **46**, an illustration of the different meanings of "meta-data" in the context of the present invention, is described in more detail. A person skilled in the art should understand that "meta-data" may refer to, but is not limited to: identification **4432**, such as filename **4436**, unique ID **4434**, version number **4440**, and hash value **4438**; authorship **4422**, such as author name **4424**, creator **4428**, contributors **4426**, and rorganization **4430**; temporal **4402**, such as creation date **4404**, last modified date **4408**, publication date **4410**, and expiration date **4404**; source **4412**, such as origin URL **4416**, database source **4420**, file path **4414**, and reference information **4418**; content **4452**, such as title **4458**, abstract **4454**, keywords **4460**, language **4456**, and genre/category **4462**; technical **4442**, such as file type **4450**, file size **4444**, character encoding

4446, and word/page count **4448**; rights **4484**, such as copyright information **4490**, license **4488**, access permissions **4492**, and usage restrictions **4486**; relational **4474**, such as parent document **4482**, related documents **4478**, position within document **4480**, and hierarchy information **4476**; contextual **4464**, such as project name **4468**, department **4472**, subject area **4466**, and intended audience **4470**; and processing **4494**, such as processing status **3800**, indexing information **3802**, classification tags **4498**, and confidence score **4496**.

(144) Referring now to FIG. **47**, an illustration of the different meanings of "document processing" or "chunk/block processing" in the context of the present invention, is described in more detail. In the context of the present invention, it should be noted that "document processing" and "chunk processing" are used in a broad and inclusive manner. A person skilled in the art should understand that "document processing" or "chunk/block processing" may refer to, but is not limited to: document manipulation **4556**, such as documents being split into chunks **4563**, and documents can being summarized into one or more chunks **4558**; chunking process **4500**, which can vary based on specific needs **4502**, for example a ten-page PDF can be broken into 5 three-page chunks (with overlapping text) **4506**, the same PDF could be divided into 5 two-page chunks **4510**, the same PDF could be trimmed and split into just three 2-page chunks **4508**, and/or, alternatively, the same PDF could be condensed by summarization into two 3-page chunks **4504**; adding metadata **4512**, where rich metadata is associated with each chunk, such as descriptive metadata about the chunk itself **4516**, relationship metadata linking the chunk to its parent document **4518**, and relationship metadata connecting the chunk to other chunks and their parent documents **4520**; grouping **4522**, where chunks and documents can be grouped based on similarities **4524** (affine chunk groups **4528** and affine document groups **4526** representing similar content) or based on differences **4530** (anti-affine chunk groups **4534** and anti-affine document groups **4532** representing different viewpoints or contrasting content; complex relationships exist within the system **4536**, such as documents linking to other documents **4538**, documents linking to their constituent chunks **4548**, chunks linking to other chunks **4550** within the same document **4552** and across different documents **4554**, and relationship types **4540** which can be inherited **4542**, derived on the fly **4546**, or captured in a graph structure **4544**.

(145) The precise definitions, sizes, characteristics, and relationships of "documents", "chunks", "blocks", "meta-data" and "document processing" may vary based on the particular embodiment, implementation, or application of the invention being described.

(146) Referring now to FIG. **48**, an illustration of an approach of generating derived prompts and using them with a RAG system or a SCORE-RAG system **4606** to generate the best results, is described in more detail. User **4600** provides a prompt **4604**. The prompt is sent to the AI Input Broker **4602** which generates multiple derived prompts. Existing RAG systems often rely on single, user-provided prompts for information retrieval. This approach can be limited by the specificity of the prompt and may not capture the full spectrum of relevant information. The present invention addresses this limitation by automatically generating a set of derived prompts that explore various aspects, phrasings, and contextual interpretations of the original query. The prompt derivation engine employs several techniques to generate derived prompts: 1. Paraphrasing: The original prompt is rephrased while maintaining its core meaning. This captures different ways of expressing the same query. 2. Query Expansion: Relevant terms or synonyms are added to the original query to broaden its scope and capture related concepts. 3. Aspect-based Prompts: The original query is broken down into different aspects or sub-topics, with separate prompts created for each. 4. Question Transformation: Statements are converted into questions and vice versa. The query is also transformed into different question types (e.g., who, what, when, where, why, how). 5. Persona-based Prompts: The query is rephrased from different persona perspectives (e.g., expert, novice, skeptic). 6. Contextual Variations: Context is added or modified to the original prompt to explore different angles. 7. Abstraction and Specification: More general and more specific versions of the original query are created.

(147) The knowledge documents **4622** are fed to a RAG or SCORE-RAG system **4606** which creates embeddings of the documents and indexes in the vector database **4624**. If a SCORE-RAG system is used, it leverages context-optimized retrieval techniques to enhance the traditional RAG approach. This system enhances the traditional RAG approach by incorporating advanced techniques such as topic modeling, intelligent document chunking, citation analysis, and hybrid search methods. Upon receiving the derived prompts **4608**, the RAG or SCORE-RAG system **4606**, proceeds with the retrieval and augmentation tasks creating refined contexts **4616**, **4618**, **4620**. These refined contexts are then transmitted to one or more LLMs **4610**, **4612**, **4614** for the generation task.
(148) Referring now to FIG. **49**, an illustration of an approach of generating derived prompts and using a LASER system **7408** and a RAG or SCORE-RAG system **4714** to generate the improved results, is described in more detail. User **4700** provides a prompt **4704**. The prompt is sent to an AI Input Broker **4702** which generates multiple derived prompts. The knowledge documents **4706** are passed through the LASER system **4708**. The LASER system **4708** employs the iterative attention focusing technique to refine and distill the context of the long document. After refinement, the refined documents **4710** fed to the RAG or SCORE-RAG system **4714** which creates embeddings of the documents and indexes in a vector database **4728**. Upon receiving the derived prompts **4712**, the RAG or SCORE-RAG system **4714**, proceeds with the retrieval and augmentation tasks creating refined contexts **4722**, **4724**, **4726**. These refined contexts are then transmitting to one or more LLMs**4716**, **4718**, **4720** for the generation task. Distinctions between the approaches in FIGS. **48** and **49** lie in the introduction of the LASER system **4710** for attention span refinement. The LASER system **4710** acts as an intermediary processor that prepares the long documents (knowledge documents **4706**) for more effective handling by the LLM. The LASER system **4710** addresses the limitations of the prior art by introducing an intelligent preprocessing step which enables LLMs to effectively handle long documents that would otherwise exceed their attention capacity.
(149) Referring now to FIG. **50**, an illustration of an exemplary set of APIs of a LASER system according to an embodiment of the invention are described in more detail. The illustration depicts APIs categorized based on modules: Document Intake **4802**; Document Splitting **4804**; Batch Processing **4806**; Model Processing **4808**; Ranking **4810**; Clustering **4812**; Iteration Control **4814**; and Result Aggregation **4816**. The Document Intake module **4802** includes APIs for document submission and document retrieval. These endpoints facilitate the ingestion and initial metadata retrieval of documents within the system. The Document Splitting module **4804** includes APIs for splitting a document and getting document blocks/chunks. The Batch Processing module **4806** includes APIs for creating a batch for processing and getting batch information. The Model Processing module **4808** includes APIs for processing a batch using the model and getting the batch output. The Ranking module **4810** includes APIs for ranking batch outputs and getting batch rankings. The Clustering module **4812** includes APIs for clustering blocks/chunks and getting document clusters. The Iteration Control module **4814** includes APIs for initiating subsequent iterations and status retrieval. These endpoints enable the system to manage and monitor the iterative refinement process. The Result Aggregation module **4816** has APIs for retrieving the refined documents, representing the final output of the long document attention span refinement process.
(150) Referring now to FIG. **51**, an illustration of an exemplary set of APIs of the SCORE-RAG system are described in more detail. The illustration depicts the APIs categorized based on modules: Document Processing **4902**; Metadata Tagging **4904**; Indexing **4906**; Query Processing **4908**; Generation **4910**; Evaluation **4912**; and System Management **4914**. The Document Processing module **4902** includes APIs for document ingestion, analysis, and pre-processing. The Metadata Tagging module **4904** includes APIs for enriching document chunks with metadata. The Indexing module **4906** includes APIs for managing the indexing of processed document chunks.

The Query Processing module **4908** includes APIs for handling user queries and search functionalities. The Generation **4910** module includes APIs for response generation. The Evaluation **4912** module includes APIs for assessing the quality of generated responses. The System Management module **4914** includes APIs for system-level operations and monitoring.

(151) The Application Programming Interfaces (APIs) described in FIG. **50** and FIG. **51** herein are not limited to any particular implementation technology or architecture. It should be understood that these APIs can be implemented using various technological approaches, including but not limited to microservices architectures, serverless computing platforms (e.g., AWS Lambda, Azure Functions, Google Cloud Functions), Function-as-a-Service (FaaS) models, containerization technologies (e.g., Docker, Kubernetes), remote procedure call frameworks such as gRPC (gRPC Remote Procedure Call), RESTful web services, GraphQL endpoints, or any combination thereof. The choice of implementation technology may depend on factors such as scalability requirements, performance needs, deployment environment constraints, and integration with existing systems. Furthermore, the APIs may be implemented using different programming languages, frameworks, and runtime environments, as appropriate for the specific use case and technological ecosystem. The flexibility in implementation allows for optimal adaptation of the system to various operational contexts while maintaining the core functionality and architectural principles described in this specification. The textbook *A First Course in Cloud-Based Microservices* (Arshdeep Bahga and Vijay Madisetti, VPT Press, 2024) is incorporated herein by reference in its entirety.

(152) Throughout the application, reference may be made to various computer hardware, including servers, GPUs, storage, cloud storage, and the like. It is contemplated and included within the scope of the invention that the LASER and SCORE-RAG systems and their various components may be software executed on computer devices, including servers, personal computers, smartphone devices, and the like, each comprising a processor configured to execute commands received from software (such as microprocessors, field-programmable gate arrays, integrated circuits, and the like), a non-transitory computer-readable storage medium positioned in electrical communication with the processor and operable to store software and other digital information thereupon in one or both of transitory and non-transitory status (such as hard disk drives, solid state drives, flash drives, compact flash drives, SD drives, memory, and the like), and a network communication device operable to communicate across computer networks as are known in the art, including, but not limited to, wide area networks such as the Internet and mobile data networks, local area networks such as Ethernet and Wi-Fi networks, and personal area networks such as Bluetooth networks. Accordingly, it is contemplated and included within the scope of the invention that the computer hardware performing the above-described LASER and SCORE-RAG systems includes hardware necessary for such performance as is known in the art.

(153) Referring now to FIG. **52** is an illustration of prefill and decode phases in an LLM inference process is described in more detail. LLM inference is the process of using a trained LLM to generate text or perform other language-related tasks. It involves two main phases; prefill **5000** and decode **5020**. The prefill phase **5000** occurs at the beginning of the inference process when the model is presented with an initial prompt **5002** or context. The prefill phase **5000** processes all input tokens **5002** in parallel, making it computationally intensive but requiring only one execution per inference session. The input **5002** undergoes tokenization **5004** to convert it into a format suitable for processing. These tokens are transformed into embeddings through an embedding lookup step **5006**. The embedded tokens pass through a series of neural network layers at step **5008**, where each layer computes query (Q), key (K), and value (V) vectors **5010**, resulting in pluralities of Q vectors, K vectors, and V vectors. These pluralities of vectors are used in a self-attention mechanism at step **5012**, followed by processing through a feed-forward network at step **5014**. After layer normalization at step **5016**, where normalized K and V vectors are produced, the normalized key and value vectors are stored in a KV cache at step **5018** for later use.

(154) The decode phase **5020** follows the prefill phase **5000** and is responsible for generating new

tokens one at a time. The decode phase **5020** is characterized by autoregressive generation, where each new token depends on all previously generated tokens. The decode phase begins by generating a new token at step **5040** based on the context provided by the prefill phase **5000**. This new token undergoes embedding lookup at step **5022** and is processed through the neural network layers at step **5024**, similar to the prefill phase **5000**. The neural network computes query (Q), key (K), and value (V) vectors **5026** for the new token at each layer, resulting in pluralities of Q, K, and V vectors. An attention computation at step **5029** is performed using the newly computed Q vector and the K and V vectors from both the KV cache **5028** (populated during the prefill phase **5000** and previous decode steps) and the newly computed K and V for the current token. The resulting output passes through a feed-forward network **5030** and layer normalization **5032**, producing normalized K and V vectors. The model then generates token probabilities **5034** and selects the next token **5036** based on these probabilities. The newly generated token's normalized K and V vectors are added to the KV cache **5038**, which grows with each new token. The decode phase **5020** iterates, using the newly generated token **5040** as input for the next cycle, until the desired output length is reached or a stop condition is met. The final output of this process is the generated text **5042**, which represents the model's response.

(155) Referring now to FIG. **52** and also referring back to FIG. **15**, the lambda architecture for h-LLMs can be further improved by incorporating prefill and decode strategies. In the batch layer **1402**, the base h-LLM can be improved to handle prefill operations more efficiently. By implementing chunked-prefills, where a prefill request is split into near-equal sized chunks, the batch layer can process large prompts more effectively. This approach allows for improved utilization of GPU resources and can improve throughput for batch processing of historical data. The real-time layer **1404** is improved by using dynamic or hybrid scheduling techniques. By allowing new requests to join a running batch without pausing ongoing decodes, the real-time layer can maintain lower latency while achieving higher throughput. This is beneficial for handling streaming data and providing quick responses to user queries. These improvements to the batch and real-time layers help in balancing the throughput-latency tradeoff by interleaving prefill and decode operations.

(156) Referring now to FIG. **52** and also referring back to FIG. **17**, the in-memory processing architecture can be further improved by leveraging High Bandwidth Memory (HBM) technologies. HBM offers higher bandwidth and lower power consumption compared to traditional DRAM, making it ideal for LLM inference workloads. In the processing layer **1606**, multiple h-LLMs can be loaded into HBM, allowing for faster access to model parameters. This is particularly beneficial for the decode phase, which requires frequent, small memory accesses. By storing frequently accessed data, such as attention keys and values (KV Cache), in HBM, the system can reduce memory access latency and improve overall inference speed. Furthermore, the batches of input data **1604** can be more efficiently managed in HBM, allowing for rapid prefill operations. The high bandwidth of HBM enables the system to quickly load and process large input prompts.

(157) Referring now to FIG. **52** and also referring back to FIG. **15** and FIG. **17**, additional memory-related optimizations can be implemented to further enhance the efficiency of LLM inference within the lambda architecture and in-memory processing framework. These enhancements focus on three areas: KV Cache Optimizations, Tiered Memory Systems, and Progressive Loading.

(158) KV Cache Optimizations are crucial for improving memory efficiency during the decode phase of LLM inference. One approach involves implementing KV Cache Pruning, where less important entries are removed from the cache based on factors such as attention scores or token positions. This pruning can be adaptive, with thresholds that adjust based on available memory and sequence length. Additionally, periodic cache cleanup can be performed to remove entries that haven't been accessed recently. Another strategy is Adaptive KV Cache Sizing, where the size of the KV cache is dynamically adjusted based on the current sequence length, available system memory, and model complexity. This can be implemented using a sliding window approach,

keeping only a fixed number of recent tokens in the cache.

(159) Tiered Memory Systems leverage different types of memory to balance performance, capacity, and cost, which is particularly relevant for large-scale LLM inference. One approach involves the integration of HBM, DRAM and Non-Volatile Memory (NVM). DRAM for frequently accessed data like the active parts of the model and the KV cache, while leveraging NVM for storing less frequently accessed model parameters. Intelligent data movement algorithms can be implemented to predict which parts of the model will be needed next and preemptively move them to DRAM. A more complex tiered caching system can be developed where different levels of cache use different memory technologies. For example, L1 Cache could use on-chip SRAM for immediate access, L2 Cache could employ High Bandwidth Memory (HBM) for high-speed, larger capacity, L3 Cache could utilize DRAM for a balance of speed and capacity, and L4 Cache could leverage NVM for large capacity. Predictive prefetching algorithms can be implemented to move data between tiers based on usage patterns and model architecture. For multi-GPU or distributed systems, heterogeneous memory management strategies can be developed to efficiently use different memory types across devices, using faster memory (e.g., HBM) on primary computation devices while offloading less critical data to devices with larger but slower memory.

(160) Progressive Loading techniques aim to optimize memory usage by loading only the necessary parts of the model as needed, rather than loading the entire model upfront. Layer-wise Progressive Loading can be implemented, where model layers are loaded into memory sequentially as they are needed for computation. For transformer models, this could mean loading encoder layers progressively for the input processing, then loading decoder layers as needed for generation. Efficient layer swapping mechanisms can be developed to manage memory when the model size exceeds available memory. Adaptive Model Pruning techniques can also be implemented, where the model size is adapted based on input complexity. For simpler inputs, a smaller, pruned version of the model can be loaded, progressively loading more complex model components as needed for challenging inputs.

(161) These memory optimization techniques, when integrated with the lambda architecture described in FIG. **15** and the in-memory processing architecture outlined in FIG. **17**, can enhance the efficiency and performance of LLM inference. By implementing these strategies, the system can better manage memory resources, reduce latency, and improve overall throughput, particularly for large-scale language models and high-volume inference tasks.

(162) Referring now to FIG. **53**, an illustration of an approach for creating, managing, and refining superchunks and using them a SCORE-RAG system according to an embodiment of the invention is described in more detail. While describing the superchunk approach, reference back to the SCORE-RAG architecture illustrated in FIG. **40** will also be made. The superchunk approach, according to an embodiment of the invention, represents an improved method for enhancing the indexing and retrieval of documents and their chunks and utilization of information in the context of RAG systems. The process begins with multiple source documents **5100**, **5102**, **5104**, each of which undergoes a chunking process to create chunks **5108**, **5110**, **5112**, **5114**, **5116**, **5118**, **5120**, **5122**, and **5124**. This chunking process divides each document into smaller, semantically coherent units referred to as chunks in certain embodiments, and in other embodiments they may correspond to paragraphs in a document (and/or images and audio content) processed in a sequential or predetermined order. Each chunk is then associated with metadata, which may include, but is not limited to, information about the parent document, creation date, category, topics, and citations or links to other documents and/or their chunks. The chunking can be performed, for example, by Document Chunking module **3806** (from FIG. **40**).

(163) The chunks may then be analyzed to evaluate the importance and relevance of each chunk. This analysis takes into account factors such as the chunk's content, its metadata, and its relationships with other chunks and documents. The Chunk Selection and Ranking module **3810** (from FIG. **40**) assigns a score or rank to each chunk based on these factors.

(164) A superchunk creator module may be additionally comprised by the SCORE-RAG system illustrated in FIG. **40**. Other embodiments may comprise a superchunk creator module as part of a LASER system/module as described above. The superchunk module may be operable to select the highest-ranked chunks from across multiple documents to form a superchunk **5106**. A superchunk may be defined as a collection of the most important and relevant chunks related to a specific topic or category, regardless of their source documents. Each superchunk may be assigned a unique identifier (UUID), which is used to manage and retrieve the superchunk. The superchunk may be created in response to a user query or a type of user query that is expected, or in response to the domain-specific nature of the Generative AI application, e.g., financial analysis or code generation, for example.

(165) The Indexing Engine **3814** (from FIG. **40**) may then process the created superchunks for efficient storage and retrieval. This indexing takes into account the composite nature of superchunks, allowing for rapid access to highly relevant information across multiple source documents. The memory access architecture (e.g., high bandwidth memories and their widths) may also be optimized for storage and retrieval of superchunks.

(166) When a user query is received, for example, the query processor **3816** (from FIG. **40**) analyzes the query to identify relevant topics or categories. The retrieval orchestrator **3818** (from FIG. **40**) then prioritizes the retrieval of superchunks that match these identified topics. This approach ensures that the most pertinent information from across multiple documents is readily available for the RAG process.

(167) The generation module **3822** (from FIG. **40**) utilizes the retrieved superchunks to generate a response to the user query. By working with superchunks, the Generation module **3822** has access to a concentrated set of relevant and/or highly relevant information, thus improving the quality and relevance of the generated response for the particular domain.

(168) The Evaluation & Fine-tuning module **3824** (from FIG. **40**) continuously assesses the performance of superchunks in query responses and refines their composition based on usage patterns and effectiveness metrics. This ongoing optimization ensures that the system adapts to changing information landscapes and user needs.

(169) The superchunk approach offers several advantages over traditional RAG systems: 1. Enhanced Relevance: By combining the most important chunks from multiple documents, superchunks provide a more comprehensive and focused information set for the RAG process. 2. Improved Efficiency: The use of superchunks reduces the volume of data that needs to be processed during query time, potentially leading to faster response generation. Superchunks can be considered as refined documents in this context. Instead of using with the original set of documents for RAG, a more efficient approach uses the superchunks. 3. Cross-Document Insights: Superchunks enable the system to draw connections and insights from across multiple documents, thus uncovering relationships that might be missed in a document-centric approach. 4. Adaptability: The continuous evaluation and optimization of superchunks allow the system to evolve and improve its performance over time. 5. Scalability: As the volume of source documents grows, the superchunk approach becomes increasingly valuable, providing a method to distill large document collections into manageable, highly relevant information units.

(170) Referring now to FIG. **54**, an illustration of the different non-limiting embodiments of “superchunk” in the context of the present invention, is described in more detail. A person skilled in the art should understand that “superchunk” may refer to, but is not limited to: a collection of pointers **5228** to individual chunks **5230** from one or more documents **5232**; a fully populated refined document **5206** composed of selected chunks **5208**; a combination **5222** of pointers to chunks **5224** along with their respective summaries **5226**; a set of chunks **5272** selected based on specific criteria such as relevance **5276**, importance **5280**, or recency **5278**; a dynamically assembled collection **5202** of information tailored to a particular query or context **5204**; a hierarchical structure **5266** of chunks **5270** and sub-chunks **5268**; a semantically related group of

chunks **5216**, regardless of their source documents; a compressed or encoded representation **5252** of multiple chunks **5254**; a versioned collection of chunks **5256** that evolves over time; a multi-modal collection **5264** incorporating text **5262**, metadata **5282**, and references to non-textual content **5260**; a unit of information **5210** optimized for specific memory access patterns **5212** or caching strategies **5214**; a privacy-enhanced collection **5234** of information with sensitive data removed or masked **5236**; a dynamically adjusted set of information **5246** based on privacy **5248** and security requirements **5250**; a content unit **5242** that can be associated with targeted advertisements **5244**; and a categorized information set **5238** that facilitates content-relevant marketing **5240**.

(171) Referring now to FIG. **55**, an illustration of the characteristics of "superchunks" in the context of the present invention is described in more detail. Superchunks may include composition characteristics **5302**. The composition of superchunks **5300** within the system can vary based on different factors and implementation choices, including, but not limited to: a) Accuracy/Relevance Based **5312**: Superchunks may be assembled based on accuracy or relevance metrics, with the potential for more accurate or comprehensive superchunks to be made available to users with higher service tiers or permissions. b) Internal Structure Variation **5314**: The internal structure of a superchunk may vary from a simple list of references to a complex data structure incorporating multiple layers of information and metadata. c) Pre-computed or Dynamic **5316**: Superchunks may be pre-computed and stored, or they may be dynamically generated in response to specific queries or user interactions. d) Machine Learning Refined **5318**: The process of creating and maintaining superchunks may involve machine learning algorithms that continuously refine the selection and organization of chunks based on usage patterns and feedback. e) Version Controlled **5320**: Superchunks may incorporate version control mechanisms to track changes over time and allow for rollback or comparison between different states. f) Adjustable Granularity **5322**: The granularity of information within a superchunk may be adjustable, allowing for different levels of detail to be presented based on user needs or system capabilities. g) Relationship Metadata **5324**: Superchunks may include metadata about the relationships between contained chunks for capturing complex networks of information. f). The memory and caches hierarchy and storage architecture interconnecting the GPUs to the high-high bandwidth memories.

(172) Superchunks may include utilization characteristics **5304**. The utilization of superchunks within the system can vary based on different factors and implementation choices, including, but not limited to: a) Domain-Specific Strategies **5326**: The system may employ different strategies for superchunk creation and utilization based on factors such as the domain of information. b) User Preference Based **5328**: The system may employ different strategies for superchunk creation and utilization based on factors such as user preferences. c) Resource-Aware **5330**: The system may employ different strategies for superchunk creation and utilization based on factors such as computational resources available. d) High-Speed Memory Cached **5332**: Superchunks may be preferentially stored in high-speed memory or caches to optimize access times and system performance. The system may employ intelligent caching strategies that prioritize frequently accessed or highly relevant superchunks for rapid retrieval.

(173) Superchunks may further comprise privacy and security characteristics **5306**. The creation and maintenance of superchunks may involve privacy protection and security mechanisms, including, but not limited to: a) PII Detection & Removal **5334**: Automatic detection and removal or masking of Personally Identifiable Information (PII) before the superchunk is used in downstream processes such as feeding to LLMs or in a RAG pipeline. b) Unlearning Techniques **5336**: Implementation of "unlearning" or "targeted catastrophic forgetting" techniques that can dynamically adjust the content of superchunks based on privacy requirements or user queries. c) Differential Privacy **5338**: Application of differential privacy techniques to add noise to sensitive data while maintaining overall statistical properties of the information. d) Content Safety Screening **5340**: Content safety screening to prevent the inclusion of explicit, violent, or otherwise

inappropriate material. e) Security Checks **5342**: Security checks to detect and remove potential malware, phishing attempts, or other security threats. f) Fact-Checking **5344**: Fact-checking to flag or filter out misinformation or unverified claims. g) Bias Detection **5346**: Bias detection and mitigation to ensure a balanced representation of information.

(174) Superchunks may further comprise monetization characteristics **5308**. Superchunks may be associated with advertising content based on their composition or categorization, including (but not limited to).: a) Content-Based Ads **5348**: Inserting relevant advertisements into superchunks based on the topical content. For example, a superchunk about natural farming practices might include or be associated with advertisements from organic fertilizer companies. b) Category-Linked Ads **5350**: Linking advertisements to the category or classification of the superchunk. For instance, a superchunk related to rent control laws in a specific city might be associated with advertisements for local legal services. c) Ad Bidding System **5352**: Implementing a bidding or auction system for advertisers to target specific types or categories of superchunks.

This advertisement feature can generate advertisements that are relevant to the user query, user interests, user intentions, or user past history of interactions. As part of the derived queries, certain queries may be made by the AI brokers to the user to identify their specific goals and intentions (for example, the AI brokers may ask the user if they are interested in buying a new car in the next six months, given that the user query appears to research and compare various brands of automobiles). The two-way interaction between the AI brokers and the user is seen as another novelty of certain embodiments of the present invention, compared to the one way interaction users currently have with generative AI LLMs.

(175) Superchunks may further comprise processing characteristics **5310** The processing of superchunks may include multiple stages of enhancement and screening, including, but not limited to: a) Privacy Enhancements **5354**: Implementation of privacy enhancements, including, detection and removal of PII, data anonymization or pseudonymization techniques. b) Ad Integration **5356**: Ad generation and integration based on content analysis and categorization. c) Safety Guardrails **5358**: Application of safety and security guardrails to filter or flag potentially harmful or inappropriate content. d) Tiered Access **5360**: The system may employ different strategies for superchunk creation, maintenance, and utilization based on factors such as user authentication level, subscription tier, or specific privacy and security requirements of the use case.

These variations and implementations of superchunks are not mutually exclusive, and the present invention incorporates systems that may include multiple approaches or allow for dynamic switching between different superchunk paradigms based on context or requirements.

(176) Referring now to FIG. **56**, an illustration of an architecture of a Hybrid-RAG system is described in more detail. A Hybrid Retrieval-Augmented Generation (Hybrid-RAG) system is designed to process and generate multi-modal data, including but not limited to text, documents, images, audio, video, and code. The system leverages a combination of various database types and LLMs to overcome the limitations of traditional Vector-RAG or Graph-RAG systems, providing enhanced performance and versatility across diverse data types and query scenarios. The Hybrid-RAG system comprises multiple components designed to efficiently process, store, retrieve, and generate multi-modal data.

(177) To achieve optimal performance for multi-modal data, Hybrid-RAG employs multiple embedding models and specialized databases, each fine-tuned for a specific content type such as text, audio, images, video, or code. This specialized approach ensures that the unique characteristics and nuances of each content modality are accurately captured and indexed. The system begins with the indexing of multi-modal data **5400**, which may include text, documents, audio, video, code, and other data types. The indexing process **5402** involves several steps: 1. Data Ingestion: Raw multi-modal data is ingested into the system. 2. Preprocessing: This step includes chunking, filtering, and cleaning of the ingested data. 3. Embedding Generation: Specialized embedding models generate vector representations for each data type.

The processed and embedded data is stored in a variety of database types **5404**, including: a Vector Databases **5406** (e.g., Pinecone, Milvus) for efficient similarity search; Graph Databases **5408** (e.g., Neo4j, TigerGraph) for relationship-based queries; Document Databases **5410** (e.g., MongoDB, Couchbase) for unstructured data; Relational Databases **5412** (e.g., PostgreSQL, MySQL) for structured data; Non-Relational Databases **5414** (e.g., DynamoDB) for unstructured or semi-structured data; Time-Series Databases **5416** (e.g., InfluxDB, TimescaleDB) for temporal data; In-Memory Databases **5418** (e.g., Redis, Memcached) for high-speed data access; Spatial/GIS Databases **5420** (e.g., PostGIS) for location-based data; Object-Oriented Databases **5422** (e.g., ObjectDB) for complex object storage; Column-Oriented Databases **5424** (e.g., Apache Cassandra) for wide-column storage; Full-Text Search Engines **5426** (e.g., Elasticsearch, Solr) for keyword-based retrieval; and Other specialized database types **5428** (e.g. NewSQL, multi-modal databases, RDF stores, XML databases, etc).

(178) When a user **5444** submits a query **5446**, the system pre-processes the query. Query Preprocessing **5448** involves filtering, embedding generation, and the creation of derived queries. Based on the preprocessed query, the system determines which database(s) are most suitable for retrieval in a query routing process. The system then queries the selected databases to retrieve relevant context **5430**. The retrieved context **5430** undergoes processing **5432** including filtering, cleaning, and ranking to generate the refined context **5434**. One or more appropriate LLMs or h-LLMs **5436** are then selected based on the query type and refined context **5434**. The responses **5438** generated by the LLMs or h-LLMs **5436** undergo filtering, cleaning, and ranking at a post-processing step **5440**. The final processed response **5442** is then delivered to the user. The previously used contexts may also be stored in-memory (for example, a cache) for faster and more accurate processing times.

(179) For the generation phase, the Hybrid-RAG utilizes an ensemble of LLMs or h-LLMs **5436**, each specialized for different tasks. These may include models optimized for question-answering, code generation, image interpretation, audio transcription, and video analysis, among others. This multi-faceted approach allows Hybrid-RAG to not only process a wide range of input types but also to generate appropriate and context-aware multi-modal outputs.

(180) The Hybrid-RAG system addresses limitations of traditional RAG systems by utilizing the most appropriate database(s) for each data type and query scenario. Hybrid-RAG enables multi-modal data processing and generation, thus providing more comprehensive and accurate responses through the integration of multiple data sources and LLMs.

(181) Referring now to FIG. **57**, an illustration of an architecture of a NoRAG system according to an embodiment of the invention is described in more detail. LLMs face limitations in accessing current information, maintaining factual accuracy, and providing transparent, attributable responses. RAG systems address these limitations of LLMs. RAG is useful for tasks requiring current or specialized knowledge, as it allows language models to draw upon external, updatable sources of information. However, RAG often introduces complexities in implementation and maintenance. Users may (depending on their design options) have to deal the complexities of chunking, embedding, indexing documents, maintaining vector databases, for instance. NoRAG system provides a novel and innovative approach to enhance LLMs without the need for RAG systems, hence the name NoRAG. By integrating key functionalities directly in a plug-in manner into the LLM architecture, in some embodiments as a license-able plugin to LLMs, NoRAG offers improved performance, reduced complexity, and enhanced user experience compared to traditional RAG systems.

(182) The NoRAG system begins with ingesting multi-modal data **5500**, which may include text, documents, images, audio, video, code, and other data types. The NoRAG system **5502** comprises several modules, each designed to perform specific functions in the overall process of enhancing LLM capabilities.

(183) The NoRAG system comprises a Document/Input Processor **5504** module.

(184) The Input Processor module **5504** is responsible for processing input documents and data sources. It handles various file formats, extracts relevant information, and prepares the data for integration into the NoRAG system.
(185) The NoRAG system further comprises a Query Processor module **5506**: The Query Processor module **5506** handles user queries, performing sophisticated analysis to improve them for the LLM **5536**. It breaks down complex queries into manageable parts and generates derived queries when necessary.
(186) The NoRAG system further comprises a Response Processor module **5508**. The Response Processor module **5508** performs post-processing on the LLM's **5536** output **5534** before sending it to the user. This module refines the generated content, ensures coherence, and applies any necessary formatting or style adjustments to enhance the quality and relevance of the final response.
(187) The NoRAG system further comprises Dynamic Knowledge Integrator component **5510**. The Dynamic Knowledge Integrator component **5510** interfaces directly with the LLM **5536**, providing relevant information during the generation process. It acts as a bridge between the LLM's **5536** inherent knowledge and the additional information processed by the NoRAG system, improving integration of external knowledge into the LLM's **5536** responses **5534**.
(188) The NoRAG system further comprises a Domain Specific Agents module **5512**: The Domain Specific Agents module **5512** comprises several domain specific agents which retrieve appropriate specialized knowledge on the query context (e.g. web search agent, stock market agent, weather data agent, loT data, etc). It enables the NoRAG system to adapt its responses to specific domains, improving accuracy and relevance in specialized fields.
(189) The NoRAG system further comprises an Internal Indexing module **5514**. The Internal Indexing module **5514** utilizes a combination of diverse database types, including, but not limited, to vector databases, graph databases, document databases, time-series databases, full-text search engines, in-memory databases, object databases, spatial databases, SQL databases, NoSQL databases, and column databases. This approach ensures efficient indexing and retrieval of information, improving the NoRAG system's performance across various data types and query patterns.
(190) The NoRAG system further comprises Specialized Domain Adapters modules **5516**: These plug-in modules **5516** contain specialized knowledge for specific domains. They can be dynamically loaded and unloaded based on the query context, allowing the NoRAG system to provide expert-level responses in various fields without overburdening the core LLM.
(191) The NoRAG system further comprises a Self-Verification system **5518**. The Self-Verification system **5518** checks facts and reduces hallucinations in the LLM's **5536** outputs **5534**. It employs internal consistency checks and compares generated content against the system's knowledge base to ensure accuracy and reliability in the responses.
(192) The NoRAG system further comprises a Source Attribution module **5520**: The Source Attribution module **5520** tracks and cites internal knowledge sources used in generating responses. It enhances the transparency and credibility of the NoRAG system's outputs by providing citations for the information used.
(193) The NoRAG system further comprises a Personalization Engine **5522**. The Personalization Engine **5522** adapts responses **5542** based on user preferences and interaction history. It maintains user profiles and adjusts the system's outputs to match individual user needs, enhancing the relevance and usefulness of the responses. This module may optionally inject advertisements in responses based on the user's subscription tier or preferences or queries sent to the user **5538** by the LLM **5536** to identify the user's attitudes, intentions, and predict behavior and future actions.
(194) The NoRAG system further comprises a Bias Detection & Mitigation module **5524**. The Bias Detection & Mitigation module **5524** identifies potential biases in the NoRAG system's responses and works to balance them. It employs advanced algorithms to recognize various types of bias and

adjusts the output to provide more neutral and fair responses.

(195) The NoRAG system further comprises a Prompt, Derived Prompts, and Context Caching module **5526**: This module **5526** caches user queries, derived prompts, and the relevant context used (including previously used contexts) that may be used to generate responses. By storing this contextual information for in-memory processing, the NoRAG system can improve response times for similar queries and maintain consistency in its outputs over time.

(196) The NoRAG system further comprises a Continuous Learning Orchestrator **5528**: The Continuous Learning Orchestrator **5528** manages the ongoing learning process of the model. It identifies knowledge gaps, prioritizes learning objectives, and coordinates the integration of new information across all modules, ensuring that the NoRAG system remains up-to-date and continues to improve over time.

(197) The NoRAG system further comprises a Security and Privacy Guardian module **5530**: The Security and Privacy Guardian module **5530** ensures data privacy and security in knowledge storage and retrieval. Privacy and security guardrails are implemented to filter sensitive data in the query and responses (such as personally identifiable information (PII)).

(198) When a user **5538** submits a query **5540**, the NoRAG system processes the query and generates a relevant context **5532** which is passed to one or more LLMs or h-LLMs **5536**. NoRAG utilizes an ensemble of LLMs or h-LLMs **5536**, each specialized for different tasks. These may include models optimized for question-answering, code generation, image interpretation, audio transcription, and video analysis, among others. The processed response **5542** is then returned to the user.

(199) The NoRAG plug-in and/or integrated LLM system may provide several advantages over traditional RAG approaches: 1. Reduced Complexity: By integrating functionalities directly into the LLM architecture, the NoRAG system eliminates the need for external retrieval systems, simplifying implementation and maintenance. The NoRAG system works like a plugin system enhancing the capabilities of an LLM. 2. Improved Performance: The tight integration of agents, domain adapters, knowledge and processing modules allows for faster response times and more coherent outputs. 3. Enhanced Customization: The modular architecture of the NoRAG system allows for easy addition or modification of specialized knowledge domains without requiring changes to the core LLM. 4. Improved Privacy and Security: By internalizing data storage and retrieval, the NoRAG system provides improved control over sensitive information and reduces potential vulnerabilities associated with external data sources. 5. Seamless Updates: The Continuous Learning Orchestrator module **5528** enables the NoRAG system to incorporate new information more efficiently than traditional RAG systems, which often require separate update processes for external knowledge bases. 6. Use in Network of LLM Agents: The NoRAG plug-in module can be used as a series or parallel network when connected to LLMs that operate as a network of LLM agents performing specialized tasks in a coordinated sequence (managed by AI brokers or LLMs, for example). Each specialized LLM agent may use a different NoRAG plugin, and NoRAG plugins may be mapped to different LLMs, depending on the type of task being done. A library of NoRAG modules may be developed in a generic manner and also with a target LLM family as an objective, and NoRAG modules for billing, advertisement generation, fault-tolerance, and security may also be added on in a plug-in manner.

(200) Referring now to FIG. **58**, an illustration of H-Token components in the FLM system according to an embodiment of the invention is described in more detail. Functional Language Modeling (FLM), a novel approach that extends traditional RAG systems by introducing Hierarchical Tokens (H-Tokens)-a higher-level abstraction that captures functional units of meaning. This approach works in conjunction with derived prompts and context optimization techniques to provide more efficient and semantically meaningful document processing. H-Tokens represent a shift in how we process and understand text in LLMs. Unlike traditional tokens that represent individual words or subwords, H-Tokens encapsulate entire functional units of meaning.

These can include: 1. Domain-specific actions (e.g., legal opinions, booking procedures) 2. Complete operational sequences (e.g., sorting algorithms, data processing routines) 3. Semantic units (e.g., menu sections, document categories)
(201) Each H-Token effectively compresses multiple regular tokens (potentially hundreds or thousands) into a single semantic unit while preserving the functional meaning of the content.
(202) The H-Token components **5600** within an FLM system include: 1. Functions **5604**: Domain-specific tasks or processes that lead to events **5610**. Functions can have multiple implementation methods. For example, booking procedures, driving operations, legal analysis. 2. Events **5606**: Outcomes or results of functions **5612**. Events serve as anchoring points for related functions. For example, reaching a destination, completing a purchase. 3. Ways **5608**: Different methods to accomplish functions **5614**. Ways can themselves be represented as H-Tokens. For example, various transportation options to reach a destination.
(203) Referring now to FIG. **59**, an illustration of steps in the FLM system according to an embodiment of the invention is described in more detail. The system begins with Input Processing **5702**, where User **5700** inputs a prompt which undergoes initial tokenization **5704**, followed by domain recognition **5706** and domain-specific analysis **5708** to understand the context and requirements of the request.
(204) The process continues with Function Identification **5710**, which comprises main function analysis **5712**, sub-function breakdown **5714**, hierarchical mapping **5716**, and function relationship analysis **5718**. This step identifies the key functional components within the input that will form the basis for H-Token generation.
(205) In the H-Token Generation phase **5720**, the system performs functional area analysis **5722**, followed by token encapsulation **5724** where regular tokens are compressed into H-Tokens. These H-Tokens are then organized hierarchically **5726**, and their relationships are mapped to maintain semantic connections **5728**.
(206) The Function Implementation stage **5730** involves H-Token mapping **5732**, function definition **5734**, event identification **5736**, and ways implementation **5738**. This phase establishes how the identified functions will be executed and relates them to specific events and implementation methods.
(207) The RAG Processing phase **5740** begins with H-Token context assembly **5742**, after which the system follows one of two processing paths **5744**. Alternative 1 involves token expansion **5746**, where H-Tokens are expanded back to regular tokens for processing. Alternative 2 utilizes direct H-Token processing **5748**. Both paths converge at context integration **5750**, where the processed information is combined into a coherent context.
(208) Finally, the Output Generation phase **5752** assembles the information **5754**, forms a response **5756**, performs quality verification **5758**, and produces the final output **5760**. This completes the FLM processing cycle, providing a semantically rich and functionally organized response to the initial user prompt.
(209) The FLM system architecture enables the system to efficiently process and understand complex inputs while maintaining semantic coherence through the use of functional abstractions represented by H-Tokens. The flexibility of choosing between token expansion and direct H-Token processing allows for optimization based on specific use cases and requirements.
(210) Referring now to FIG. **60**, an illustration of FLM system implementation in the travel domain according to an embodiment of the invention is described in more detail. The figure demonstrates how the system processes a travel-related query through various processing stages.
(211) In the Input Processing stage, the system receives a user **5800** prompt requesting to “Plan a week-long vacation to Hawaii”. This prompt undergoes initial tokenization **5804** and is recognized as belonging to the Travel domain **5806** through domain recognition algorithms.
(212) The Function Identification stage **5808** breaks down the travel planning function **5810** into three primary sub-functions: Transportation **5812**, Accommodation **5814**, and Activities **5816**. Each

of these sub-functions represents a crucial component of the travel planning process.
(213) In the H-Token Generation stage **5818**, each sub-function is further decomposed into specific H-Tokens. The Transportation function **5812** generates H-Token for Flight_Booking **5820**. The Accommodation function produces H-Token for Hotel_Booking **5822**. The Activities function creates H-Token for Beach_Activities **5824**. A sub-function can generate one or more H-Tokens. For example, Activities function can create H-Tokens for Beach_Activities, Sightseeing and Dining.
(214) The Function Implementation stage **5826** demonstrates how individual H-Tokens are processed and broken down into their respective functions, events, and implementation methods. For example, the Flight_Booking H-Token **5820** comprises: Function: Search_Flights **5828**; Event: Flight_Booked **5830**; andWays: Airlines/Dates/Routes **5832**. Similarly, the Hotel_Booking H-Token **5822** comprises: Function: Search_Hotels **5834**; Event: Room_Reserved **5836**; and Ways: Hotels/Locations/Prices **5838**. Similarly, the Beach_Activities H-Token **5824** comprises: Function: Activity_Planning **5840**; Event: Activities_Scheduled **5842**; and Ways: Tours/Self-Guided/Groups **5844**.
(215) The RAG Processing stage assembles the H-Token context **5848**, incorporating Location Context, Duration Context, and Preference Context. This context undergoes processing through either Alternative 1 (expansion to regular tokens **5854**) or Alternative 2 (direct H-Token processing **5856**), leading to either Traditional RAG **5858** or H-Token Aware Response generation **5860**.
(216) Finally, in the Output Generation stage **5862**, both processing paths converge to create a detailed itinerary **5864**, which is then formatted into the final response for the user **5866**. This processing flow enables the system to generate semantically rich and functionally organized travel plans that account for all necessary aspects of the vacation planning process.
(217) Referring now to FIG. **61**, a flow chart of a multi-pass process for a RAG system according to an embodiment of the invention is described in more detail. The process begins when User **5900** submits a query **5902** to the system. Upon receiving the query, the system initiates a hybrid document retrieval process at step **5904**. The hybrid document retrieval process **5904** comprises two parallel operations: a keyword search **5906** performed on a relational database and a vector search **5908** performed on a vector database using query embeddings. The results from both search operations, taking the form of the identification or documents and/or content comprised by documents comprised by the respective relational and vector databases, are merged at step **5910** to form a combined context **5912** containing the most relevant documents. Additionally, the search can also be done on the Web at step **5904**. Results from the web search at step **5904** may entirely replace results found at **5906** and **5908** and/or may be supplementary to those results. The combined context is then split into a plurality of partitions at step **5914**. Splitting the combined context into multiple partitions may be performed while preserving document boundaries to maintain semantic coherence. The system then enters a map phase **5916** where a plurality of mapper instances (**5924**, **5926**, **5928**) process the context partitions (**5918**, **5920**, **5922**) in parallel. Each mapper instance **5924**, **5926**, **5928** may be different instances of the same LLM or may be instances of different LLMS. Each mapper instance **5924**, **5926**, **5928** receives a partition of the plurality of partitions and a mapper prompt that includes specific instructions for analyzing the context comprised by the partition, identifying key information, extracting relevant quotes, and noting significant details. In some embodiments, the mapper instances **5924**, **5926**, **5928** may be operable to generate a prompt responsive to the received partition and, in some embodiments, additionally responsive to the query **5902**. The plurality of mapper instances **5924**, **5926**, **5928** generate intermediate analysis results **5930**, with each result containing extracted information and a calculated confidence score. The intermediate results **5930** are collected and enter a reduce phase **5932** is commenced. The system first weights the intermediate results at step **5934** based on their respective confidence scores. The weighted results **5934**, along with a conversation history **5936**, are then passed to a reducer component **5938**. The conversation history **5936** may be a dynamically

maintained log of queries and responses with the user **5900**. The reducer component **5938**, which may utilize a specialized LLM, synthesizes the weighted results **5934** into a coherent response while identifying common themes and resolving any conflicts between the intermediate results **5930**. The reduced instance **5938** may produce a coherent response that is internally coherent and coherent with the conversation history **5936**. A synthesized final response **5940** may then be returned to the user **5900**.

(218) Referring now to FIG. **62**, a flow chart of a multi-pass process for a RAG system with caching and feedback loop according to an embodiment of the invention is described in more detail. The process begins when User **6000** submits a query **6002** to the system. Upon receiving the query, the system first performs a cache check at step **6004** to determine if similar queries have been processed previously. If there is a cache hit **6006**, indicating the system has previously processed a similar query, the system retrieves the cached intermediate results from a system cache for further processing based on the user query **6002**. In case of a cache miss **6008**, where a similar query has not been processed and/or is not stored in the system cache, the system initiates a hybrid document retrieval process at step **6010**. The hybrid document retrieval process **6010** comprises two parallel operations: a keyword search performed on a relational database, and a vector search performed on a vector database using query embeddings as described for **5906** and **5908** of FIG. **61**. Additionally, the search can also be done on the Web at step **6010**, again in either a supplementary capacity of as a replacement. The results from both search operations are merged to form a combined context containing the most relevant documents.

(219) The combined context is then split into a plurality of context partitions at step **6012** while preserving document boundaries to maintain semantic coherence. The system then enters a Map Phase at step **6014**, where a plurality of mapper instances process the plurality of context partitions, either in parallel or in series. Each mapper instance receives a context partition of the plurality of context partitions and a mapper prompt that includes specific instructions for analyzing the context, identifying key information, extracting relevant quotes, and noting significant details. The mapper instances generate intermediate analysis results, with each result containing extracted information from the context partition it received and a calculated confidence score.

(220) The intermediate results are collected and the system enters a Reduce Phase at step **6016**. In the Reduce Phase **6016**, the system first weights the intermediate results based on their respective confidence scores. The weighted results, along with the conversation history, are then passed to a reducer component. The reducer component, utilizing one or more LLMs or LLM agents, synthesizes the weighted analyses into a coherent response while identifying common themes and resolving any conflicts between the intermediate results.

(221) The output of the reduce phase is processed by Guardrails at step **6018**, to ensure the response complies with the system policies, to protect against unauthorized access, privacy breaches, and to protect against potential jailbreaking attempts. The output from the Guardrails **6018** is an aggregated response **6020** which undergoes a further validation and filtering process at step **6022** where it is checked for factual accuracy, completeness, and consistency. If any issues are detected at step **6022**, the system sends a feedback refinement instruction **6026** to one or more of the earlier steps (**6004**, **6010**, **6012**, **6014**, **6016**) to refine the response. If no issues are detected at step **6022**, the validated response is then provided to the user at step **6024**. Additionally, the system updates the system cache with the intermediate results to optimize future related queries. One or more implementation tools and/or resources **6028**, including, but not limited to, LLM agents, local LLMs, cloud-based LLMs, and specialized LLM models may be employed in any or all of the map phase **6014**, the reduce phase **6016**, the guardrails **6018**, and processing the response **6022**

(222) The Multi-Pass MapReduce-based approach enables comprehensive analysis of large contexts while maintaining efficiency through parallel processing and caching mechanisms. The multi-pass architecture allows for scalable processing of document contexts, while the validation phase ensures response quality and accuracy.

(223) It should be understood that while specific implementations and features of the multi-pass approach have been described herein, the present invention is not limited to these particular embodiments. The multi-pass approach may incorporate various additional features and optimizations, including but not limited to: 1. Caching mechanisms for storing and retrieving intermediate analysis results; 2. Resource monitoring systems for tracking computational capacity and memory usage; 3. Dynamic context partitioning based on available system resources and document characteristics; 4. Scoring and weighting mechanisms for evaluating intermediate results; 5. Deployment of multiple specialized large language models optimized for different processing phases; 6. Parallel processing architectures for concurrent analysis of context portions; 7. Adaptive prompt generation systems; 8. Conversation history management; 9. Validation and verification processes; 10. Hybrid search implementations combining multiple search methodologies, and database sources; 11. Source attribution and citation tracking; 12. Error detection and correction mechanisms; 13. Feedback mechanisms whereby aggregated and intermediate results are fed back to the mapper, partitioner, and searcher components through various feedback loops and iterative evaluation processes; 14. Tasks may be performed by autonomous agents and brokers, which can operate independently or in coordination to optimize system performance; 15. Different components of the system may utilize different large language models, including but not limited to local LLMs, cloud-based LLMs, or specialized models optimized for specific tasks; 16. Synthetic data generation and training methodologies may be used to enhance the performance of mapper, searcher, and reducer components; 17. Security and privacy guardrails may be implemented prior to aggregation phases to protect against unauthorized access, privacy breaches, and potential jailbreaking attempts; 18. Historical information, cached contexts, and prior scoring approaches from SCORE-RAG implementations may be incorporated to enhance response generation and evaluation; 19. LLM agents may participate in scoring, feedback, and evaluation processes, maintaining historical scores and results for future reference and system optimization; 20. The aggregated responses may undergo additional processing by specialized agents focused on ethical considerations, sentiment analysis, and emotional modulation to generate more personalized and contextually appropriate outputs. These agents may adjust response characteristics based on user preferences, conversation history, and contextual requirements; 21. The system may maintain and utilize various types of historical data, including but not limited to: past scores and evaluations, successful response patterns, user interaction histories, and performance metrics across different system components. This historical data may be used to inform future processing decisions and optimize system performance through continuous learning and adaptation; and 22. The feedback loops and iterative processes may extend across multiple system components, allowing for dynamic optimization of search strategies, context partitioning approaches, mapping methodologies, and reduction techniques.
(224) The specific features and implementations described herein are intended to be illustrative rather than restrictive, and various modifications, combinations, and adaptations may be made without departing from the scope of the invention.
(225) Referring now to FIG. **63**, a mapper instance used in a multi-pass process for a RAG system according to an embodiment of the invention is described in more detail. A mapper instance **6106** comprises a context partition receiver **6108**, a mapper prompt generator **6110**, and a mapper LLM interface **6112**. The context partition receiver **6108** accepts a context partition **6104** (segmented portions of the combined context while maintaining document boundaries and semantic coherence). Each context partition **6104** typically contains one or more complete documents or document segments, with partition sizes dynamically adjusted based on system resources and processing requirements. The mapper prompt generator **6110** accepts the User Query **6102** and one or more context partitions from the context partition receiver **6108** and creates specialized prompts for each context partition **6104**. The mapper LLM interface **6112** is configured to at least one of operate an instance of one or more LLMs transmit the mapper prompt generated by the mapper prompt

generator **6110** to one of more LLMs and receive responses from the one or more LLMs. The mapper LLM interface **6112** can be configured to use different language models based on requirements, such as: 1. Fast processing LLM (e.g., smaller models optimized for speed); 2. Deep analysis LLM (e.g., larger models for complex reasoning); and 3. Specialized LLMs (e.g., models trained for legal or medical text analysis).

(226) Mapper instances **6106** produce intermediate results **6114**, comprising key points **6116**, quoted evidence **6118**, relationships **6120** and confidence scores **6122**. The intermediate results **6114** may be generated by an LLM comprised by the mapper instance **6106** or by an LLM that is accessed by the mapper instance **6106** via the mapper LLM interface **6112**.

(227) Referring now to FIG. **64**, a reducer instance used in a multi-pass process for a RAG system according to an embodiment of the invention is described in more detail. A reducer instance **6208** comprises, and in some embodiments may consist of, an intermediate results aggregator **6210**, a reducer prompt generator **6212**, and a reducer LLM interface **6214**. The intermediate results aggregator **6210** collects and pre-processes the outputs from multiple mapper instances **6206**, organizing them for coherent synthesis. The reducer prompt generator **6212** accepts the original query **6204**, a conversation history **6202**, and the intermediate results aggregated by the intermediate results aggregator **6210**. The reducer prompt generator **6212** creates one or more prompts to analyze the intermediate results from the mapper instances **6206**, identify common themes, resolve conflicts and create coherent responses. The reducer LLM interface **6214** is configured to at least one of operate an instance of one or more LLMs transmit the mapper prompt generated by the reducer prompt generator **6212** to one of more LLMs and receive responses from the one or more LLMs. The reducer LLM interface **6214** can be configured to use different language models based on requirements, such as: 1. Fast processing LLM (e.g., smaller models optimized for speed); 2. Deep analysis LLM (e.g., larger models for complex reasoning); and 3. Specialized LLMs (e.g., models trained for legal or medical text analysis)

(228) The reducer instance **6208** produces one or more aggregates response **6216** comprising one or more of a synthesized answer **6218**, one or more source citations **6220**, a confidence level **6222**, and one or more related questions **6224**.

(229) Referring now to FIG. **65**, an illustration of a probabilistic causal approach for a multi-pass RAG system, according to an embodiment of the invention, is described in more detail. The process begins when User **6300** submits a query **6302** to the system. Upon receiving the query, the system initiates a hybrid document retrieval process at step **6304**. The hybrid document retrieval process **6304** comprises two parallel operations: a keyword search at step **6306** performed on a relational database, and a vector search at step **6308** performed on a vector database using query embeddings. The results from both search operations **6306**, **6308** are merged at step **6310** to form a combined context **6312** containing the most relevant documents. Additionally, the search can also be done on the Web at step **6304**. The context is then split into a plurality of partitions at step **6314** while preserving document boundaries to maintain semantic coherence.

(230) The system then enters a map phase (designated as probabilistic causal extraction) at step **6316**, where a plurality of mapper instances **6318**, **6320**, **6322** process the context partitions of the plurality of context partitions in parallel. Each mapper instance **6318**, **6320**, **6322** performs event extraction within the context partition of the plurality of context partitions assigned thereto. For example, Mapper-**1** Event Extraction **6324** identifies Event-**1** **6330** and Event-**2** **6332** with a causal probability of P.sub.1 **6334** between them, Mapper-**2** Event Extraction **6326** identifies Event-**3** **6336** and Event-**4** **6338** with a causal probability of P.sub.2, **6340** and Mapper-N Event Extraction **6328** identifies Event-M **6342** and Event-M+1 **6344** with a causal probability of P.sub.n **6346**. The probability scores P.sub.1 **6334**, P.sub.2 **6340**, and P.sub.n **6346** represent the likelihood of causal relationships between sequential events within each partition and also across a sequence of multiple events being present in a particular causal order. For example, in a sequence of events **1**, **2**, **3**, **4**, **5**, **7**, and **8**, events **5** and **7** may depend on events **1** and **2** and not on **3**, but **8** depends on **3**, **5**, and **7**,

so there may be some causal dependence between certain subsets of the event sequence and not between other events, but an overall sequence may still be termed an event sequence, even If there may be causality between some subsets of sequences of events. Also, event **8** may depend on event **1** only, and not on other events. Furthermore, some events may occur at the same time in response to earlier events. However, from a logical point of a view an order of events that represent this soft causality may also be represented with a probability of occurrence to assist in multi-step reasoning tasks. An optimization algorithm in the reducing step, for instance, can help analyze parallel hypotheses based on multiple possible causal sequences of events, with each event being associated with a probability or probability distribution or measure, so that reasoning may move forward even with incomplete or unknown information at the current time. Additionally, a risk analysis may be proposed as a result of the reasoning processes. Probabilistic analyses, such as Markov analyses, may also be performed to derive probabilities, similar to Bayesian and other types of analyses, including machine learning or use of LLMs, known in probability theory and practice to carry out hypotheses testing processes.
(231) Some of the events may be fictitious events and/or event sequences that are potential or likely events, in past and/or future times, that could have occurred and are being proposed by the LLM as potential explanation of certain actions as part of the generative AI and reasoning processes. Some events may be created by users or LLMs to stimulate the creation of what/if plausible scenarios through use of LLMs. Further, some of the events may depend on events that are in the far off memory (not near term memory) as in previous event sequences used at some point in the past. Therefore, it is contemplated to assume that each event in an event sequence as described here may itself be an event sequence. This hierarchy allows a sequence of events to depend on another sequence of events, even if some of the events in each sequence may be different from the events in the depending on dependent sequence, and further multiple such probabilities may be derived and updated based on incrementally refined reasoning analyses.
(232) The outputs from all mappers feed into the reduce phase **6348** (designated as probabilistic causal analysis). Within this phase, a reducer component **6350** processes the intermediate results through a causal analysis subsystem **6352** comprising several operations: constructing causal chains **6354**, which identifies potential cause-effect sequences across all extracted events; calculating chain probabilities **6356**, which computes compound probability scores for each identified causal chain; and extracting supporting evidence **6358**, which collects relevant quotes and context supporting each causal relationship.
(233) The output of the reduce phase is an aggregated response **6360** comprising a formal response **6362** made up of a plurality of potential causal chains **6364**, **6366**, **6368**, each with an associated probability score. This multi-chain output with probability scores enables the system to represent multiple possible causal interpretations rather than committing to a single explanation. This probabilistic causal architecture enhances the basic multi-pass RAG system by incorporating uncertainty quantification throughout the processing pipeline, enabling more nuanced and reliable causal reasoning for complex queries requiring cause-effect analysis.
(234) Referring now to FIG. **66**, a flow chart illustrating the processing of a legal analysis query by a probabilistic causal RAG system according to an embodiment of the invention is presented. The process begins in a query processing phase **6400**, where the user's query **6402**—"*Analyze Judge Alan's mistrial evolution*" is analyzed at step **6404** to extract key elements **6406**, including the primary entity (Judge Alan), the action of interest (mistrials), and the analytical focus (evolution over time).
(235) The system then enters the document search phase **6408**, where both keyword-based searches in a relational database **6410** and embedding-based searches in a vector database **6412** are performed in parallel. The results from the searches **6410**, **6412** are merged at step **6414** to form a combined context **6416** containing relevant case documents, court orders, and legal analyses, as well as any other content relevant to the user query **6402**.

(236) The combined context is then split into a plurality of partitions at step **6418** while preserving document boundaries to maintain semantic coherence. The system then enters a map phase (designated as probabilistic causal extraction) at step **6420**, where a plurality of mapper instances **6422**, **6424**, **6426** process the plurality of context partitions in parallel. Each mapper instance **6422**, **6424**, **6426** extracts specific events related to the query, in this example, mistrial events, and identifies potential causal relationships. For example, Events Group-**1 6428** processes cases from the years 2015-2017 involving jury misconduct and evidence issues, identifying events 2015 Mistrial Case A **6434** and 2016 Mistrial Case B **6436** with a causal link probability **6438** of 0.85 based on similar reasoning patterns. Events Group-**2 6430** processes cases from years 2018-2020 focused on patent eligibility issues, identifying events 2018 Mistrial Case C **6440** and 2019 Mistrial Case D **6442** with a stronger causal link probability **6444** of 0.92 due to direct case references. Events Group-**3 6432** processes cases from years 2021-2024 concerning expert testimony standards, identifying events 2021 Mistrial Case E **6446** and 2022 Mistrial Case F **6448** with a causal link probability **6450** of 0.78 based on similar legal standards.
(237) The system then enters the reduce phase (designated as probabilistic causal analysis) at step **6452**. The reduce phase employs a reducer **6454** to synthesize the temporal and causal patterns identified across all event groups. The reducer **6454** performs several operations: identifying evolution patterns in the judge's decision-making at step **6458**; calculating probability scores for each pattern at step **6460**; and extracting supporting legal precedents at step **6462**.
(238) The output of the reduce phase is the aggregated response **6464**, comprising a structured analysis showing several evolution patterns in Judge Alan's mistrial decisions: evolution pattern-**1 6468** with a focus on procedural issues (years 2015-2017, probability 0.85); evolution pattern-**2 6470** with an emphasis on patent eligibility (years 2018-2020, probability 0.92); and evolution pattern-**3 6472** with development of expert testimony standards (years 2021-2024, probability 0.78). Each pattern **6468**, **6470**, **6472** is supported by evidence **6474** including relevant case citations, direct quotes, and statistical trends demonstrating the evolution of legal reasoning.
(239) Referring now to FIG. **67**, a flow chart illustrating the processing of a stock market crash by a probabilistic causal RAG system according to an embodiment of the invention is presented. The process begins in the query processing phase **6500**, where the user's query **6502** "Analyze Stock Market Crash Impact" is analyzed at step **6504** to extract key elements **6506**, including the primary event (stock market crash), the analytical focus (causal effects), and the temporal scope (sequential impact).
(240) The system then enters the document search phase **6508**, where both keyword-based searches in a relational database **6510** and embedding-based searches in a vector database **6512** are performed in parallel. The results from the searches **6510**, **6512** are merged at step **6514** to form a combined context **6516** containing market data, historical records, and financial analyses, and any other content that is relevant to the query **6502**.
(241) The combined context is then split into a plurality of partitions at step **6518** while preserving document boundaries to maintain semantic coherence. The system then enters the map phase (designated as probabilistic causal extraction) at step **6520**, where a plurality of event groups are processed in parallel. Each group extracts specific events and identifies potential causal relationships. Events Group-**1 6522** processes the chain involving interest rates, where a stock market crash **6526** leads to increased interest rates **6528** (with probability of 0.7), which causes housing prices to drop **6530** (with probability of 0.9), which leads to a surge in foreclosures **6564** (with probability of 0.8). This chain is supported by evidence **6532** comprising historical correlations, established rate hike patterns and historical foreclosure trends. Events Group-**2 6524** processes the chain involving market impact on jobs, where the stock market crash **6534** leads to loss of jobs **6536** (with probability of 0.6), leading to a surge in foreclosures **6538** (with probability of 0.7). This chain is supported by evidence **6540** comprising market impact on jobs and historical foreclosure trends.

(242) The system then enters the reduce phase (designated as probabilistic causal analysis) at step **6542**. The peduce phase comprises several operations: 1. Identify Event Patterns **6548**: Synthesizes the temporal and causal patterns identified across all event groups; 2. Calculate Pattern Probabilities **6550**: Computes compound probability scores for each identified chain; and 3. Extract Supporting Evidence **6552**: Collects relevant market data and historical correlations supporting each causal relationship.
(243) The output of the reduce phase is the aggregated response **6554**, having a formal response **6556** that, in this embodiment, comprises two causal chains: 1. Chain-**1 6558** demonstrating how the market crash influences housing prices through interest rate mechanisms, leading to foreclosures (chain probability 0.504); and 2. Chain-**2 6560** showing market impact on jobs, leading to foreclosures (chain probability 0.42).
(244) Each chain is supported by comprehensive evidence **6562** including historical data, market correlations, and financial metrics, providing a robust foundation for understanding the potential causal relationships in the stock market scenario.
(245) The specific implementations, features, components, methods, embodiments, and variations described in the above description are intended to be illustrative rather than limiting. Various modifications, combinations, adaptations, substitutions, additions, and variations can be made to the described embodiments of the probabilistic causal approach to RAG systems without departing from the scope of the invention. The Invention may incorporate additional methods of event extraction, alternative probability calculation techniques, different causal chain construction algorithms, varied mapping and reduction strategies, alternative search methodologies, and other enhancements to the core probabilistic causal framework. Different types of language models, databases, and computational architectures may be employed within the scope of the invention. The system may be implemented using various programming languages, frameworks, and platforms. The invention may be extended to include additional types of causal analysis, different probability distribution models, alternative evidence collection methods, varied validation approaches, and other mechanisms for improving causal reasoning capabilities. The system may incorporate caching mechanisms, feedback loops, and iterative refinement processes. Integration with external systems, tools, and data sources may be accomplished through different interfaces and protocols while remaining within the scope of the invention. The specific examples, use cases, and applications discussed are not intended to be exhaustive or to limit the invention to the precise forms disclosed. Further, embodiments discussed above may also be combined with each other.
(246) Some of the illustrative aspects of the present invention may be advantageous in solving the problems herein described and other problems not discussed which are discoverable by a skilled artisan.
(247) While the above description contains much specificity, these should not be construed as limitations on the scope of any embodiment, but as exemplifications of the presented embodiments thereof. Many other ramifications and variations are possible within the teachings of the various embodiments. While the invention has been described with reference to exemplary embodiments, it will be understood by those skilled in the art that various changes may be made and equivalents may be substituted for elements thereof without departing from the scope of the invention. In addition, many modifications may be made to adapt a particular situation or material to the teachings of the invention without departing from the essential scope thereof. Therefore, it is intended that the invention not be limited to the particular embodiment disclosed as the best or only mode contemplated for carrying out this invention, but that the invention will include all embodiments falling within the scope of the appended claims. Also, in the drawings and the description, there have been disclosed exemplary embodiments of the invention and, although specific terms may have been employed, they are unless otherwise stated used in a generic and descriptive sense only and not for purposes of limitation, the scope of the invention therefore not being so limited. Moreover, the use of the terms first, second, etc. do not denote any order or

importance, but rather the terms first, second, etc. are used to distinguish one element from another. Furthermore, the use of the terms a, an, etc. do not denote a limitation of quantity, but rather denote the presence of at least one of the referenced item.

(248) Thus the scope of the invention should be determined by the appended claims and their legal equivalents, and not by the examples given.

(249) The claims in the instant application are different than those of the parent application or other related applications. Applicant therefore rescinds any disclaimer of claim scope made in the parent application or any predecessor application in relation to the instant application. Any such previous disclaimer and the cited references that it was made to avoid, may need to be revisited. Further, any disclaimer made in the instant application should not be read into or against the parent application.

Claims

1. A method of processing large contexts in a retrieval-augmented generation (RAG) system comprising: receiving a query from a user; retrieving a plurality of relevant documents from at least one database based on the query, wherein the relevant documents form a combined context; partitioning the combined context into a plurality of context partitions; generating a plurality of intermediate analysis results by: processing each context partition of the plurality of context partitions using a mapper prompt; and sending an output of the mapper prompt from each context partition to one or more large language models (LLMs); generating a final response by processing the plurality of intermediate analysis results using a reducer prompt sent to one or more LLMs; and transmitting the final response to the user.

2. The method of claim 1 wherein retrieving the plurality of relevant documents comprises: identifying a first set of documents by performing a keyword search in a relational database; identifying a second set of documents by performing a vector search in a vector database; and combining the first and second sets of documents to form the plurality of relevant documents.

3. The method of claim 1 wherein processing each context partition comprises: processing the plurality of context partitions in parallel using a plurality of LLM instances; and generating the plurality of intermediate analysis results simultaneously.

4. The method of claim 1 wherein combining the plurality of intermediate analysis results comprises: identifying one or more common themes across the plurality of intermediate analysis results; resolving any conflicts between the plurality of intermediate analysis results; synthesizing the plurality of intermediate analysis results into a coherent response; and validating the coherent response for at least one of accuracy and completeness.

5. The method of claim 4 further comprising processing the coherent response for at least one of: compliance with one or more system policies; protection from unauthorized access; protection from privacy breaches; or protection from potential jailbreaking attempts.

6. The method of claim 1 further comprising: storing the plurality of intermediate analysis results in a cache; receiving a subsequent query related to the original query; retrieving the stored plurality of intermediate analysis results from the cache; and generating a new final response using the stored plurality of intermediate analysis results.

7. The method of claim 1 wherein processing each context partition comprises: extracting key information from the context partition; generating a summary of the context partition; identifying one or more relevant facts and relationships within the context partition; and storing the extracted key information, the summary, and the identified one or more relevant facts and relationships as part of the intermediate analysis results.

8. The method of claim 1 wherein combining the plurality of intermediate analysis results comprises: assigning a confidence score to each intermediate analysis result of the plurality of intermediate analysis results; generating a weighted plurality of intermediate analysis results by weighting each intermediate analysis result of the plurality of intermediate analysis results based on

their respective confidence scores; and generating the final response based on the weighted plurality of intermediate analysis results.

9. The method of claim 1 further comprising: monitoring system resources during processing; dynamically adjusting a size of the context partitions of the plurality of context partitions based on available system resources; and scheduling the processing of the plurality of context partitions to optimize resource utilization.

10. The method of claim 1 wherein: each context partition of the plurality of context partitions is processed using a first LLM optimized for analysis and extraction; the plurality of intermediate analysis results are combined using a second LLM optimized for synthesis and summarization; and the final response is validated using a third LLM that is optimized for fact-checking and verification.

11. The method of claim 1 wherein the mapper prompt comprises at least one of: instructions for analyzing the context partition; tags for structuring the analysis of the context partition; guidelines for extracting and formatting relevant information; a context section containing the context partition; and a question section containing the query.

12. The method of claim 1 wherein the reducer prompt comprises at least one of: instructions for combining the intermediate analysis results; tags for structuring the final response; guidelines for generating related questions; a context section containing the combined intermediate analysis results; a conversation history section; and a question section containing the query.

13. The method of claim 1 wherein processing each context partition using the mapper prompt comprises at least one of: identifying key information relevant to the query; extracting relevant quotes with their sources; identifying significant details, dates, and relationships; identifying insights from the context partition; and generating the intermediate analysis results within specified tags.

14. The method of claim 1 wherein processing the plurality of intermediate analysis results using the reducer prompt comprises at least one of: generating a detailed answer addressing the query; generating a list of related questions for future exploration; formatting the final response with specified tags; including source documentation references; and validating the response against provided guidelines.

15. The method of claim 1 wherein the mapper prompt and reducer prompt each include one or more guidelines comprising at least one of: guidelines for validating document relevance based on mentioned entities; guidelines for providing relevant excerpts with source attribution; guidelines for ignoring irrelevant documents; guidelines for declining to answer when no relevant context is found; guidelines for maintaining consistent terminology; and guidelines for acknowledging limitations of generating the final response.

16. The method of claim 1 further comprising: maintaining a conversation history related to the user; adding the query to the conversation history; including the conversation history in the reducer prompt; and generating the final response based upon each of the intermediate analysis results and the conversation history.

17. The method of claim 1 wherein the mapper prompt is sent to a plurality of instances of a single LLM.

18. The method of claim 1 wherein the mapper prompt is sent to a plurality of LLMs.

19. A retrieval-augmented generation (RAG) system configured to process large contexts comprising: a processor; a communication device positioned in communication with the processor and configured to send and receive transmissions across a digital network; and a storage medium having stored thereon a document database; a non-transitory computer-readable storage medium having stored thereon software that, when executed by the processor, is operable to: receive a query from a user; retrieve a plurality of relevant documents from the document database based on the query, wherein the relevant documents form a combined context; partition the combined context into a plurality of context partitions; generate a plurality of intermediate analysis results by:

processing each context partition of the plurality of context partitions using a mapper prompt; and sending an output of the mapper prompt from each context partition to a first large language model (LLM); generate a final response by processing the plurality of intermediate analysis results using a reducer prompt sent to at least one of the first LLM and a second LLM; and transmit the final response to the user.

20. The RAG system of claim 19 wherein the software is further operable to, when executed by the processor, retrieve the plurality of relevant documents by: identifying a first set of documents by performing a keyword search in a relational database; identifying a second set of documents by performing a vector search in a vector database; and combining the first and second sets of documents to form the plurality of relevant documents.

21. The RAG system of claim 19 wherein the software is further operable to, when executed by the processor, process each context partition by: processing the plurality of context partitions in parallel using a plurality of LLM instances; and generating the plurality of intermediate analysis results simultaneously.

22. The RAG system of claim 19 wherein the software is further operable to, when executed by the processor, combine the plurality of intermediate analysis results by: identifying one or more common themes across the plurality of intermediate analysis results; resolving any conflicts between the plurality of intermediate analysis results; synthesizing the plurality of intermediate analysis results into a coherent response; and validating the coherent response for at least one of accuracy and completeness.

23. The RAG system of claim 22 wherein the software is further operable to, when executed by the processor, process the coherent response for at least one of: compliance with one or more system policies; protection from unauthorized access; protection from privacy breaches; or protection from potential jailbreaking attempts.

24. The RAG system of claim 19 wherein the software is further operable to, when executed by the processor: store the plurality of intermediate analysis results in a cache; receive a subsequent query related to the query; retrieve the stored plurality of intermediate analysis results from the cache; and generate a new final response using the stored plurality of intermediate analysis results.

25. The RAG system of claim 19 wherein the software is further operable to, when executed by the processor, process each context partition by: extracting key information from the context partition; generating a summary of the context partition; identifying one or more relevant facts and relationships within the context partition; and storing the extracted key information, the summary, and the identified one or more relevant facts and relationships as part of the intermediate analysis results.

26. The RAG system of claim 19 wherein the software is further operable to, when executed by the processor, combine the plurality of intermediate analysis results by: assigning a confidence score to each intermediate analysis result of the plurality of intermediate analysis results; generating a weighted plurality of intermediate analysis results by weighting each intermediate analysis result of the plurality of intermediate analysis results based on their respective confidence scores; and generating the final response based on the weighted plurality of intermediate analysis results.

27. The RAG system of claim 19 wherein the software is further operable to, when executed by the processor: monitor system resources during processing; dynamically adjust a size of the context partitions of the plurality of context partitions based on available system resources; and schedule the processing of the plurality of context partitions to optimize resource utilization.

28. The RAG system of claim 19 wherein the software is further operable to, when executed by the processor: process each context partition of the plurality of context partitions using the first LLM, the first LLM being optimized for analysis and extraction; combine the plurality of intermediate analysis results using the second LLM, the second LLM being optimized for synthesis and summarization; and validate the final response using a third LLM that is optimized for fact-checking and verification.

29. The RAG system of claim 19 wherein the software comprises instructions for the mapper prompt to comprise at least one of: instructions for analyzing the context partition; tags for structuring the analysis of the context partition; guidelines for extracting and formatting relevant information; a context section containing the context partition; and a question section containing the query.

30. The RAG system of claim 19 wherein the software comprises instructions for the reducer prompt to comprise at least one of: instructions for combining the intermediate analysis results; tags for structuring the final response; guidelines for generating related questions; a context section containing the combined intermediate analysis results; a conversation history section; and a question section containing the query.